

REPUBLIQUE DU SENEGAL



Un peuple-Un but-Une foi
Ministère de l'Enseignement Supérieur et de la Recherche
Direction Générale de l'Enseignement Supérieur Privé
Institut Supérieur d'Informatique

ISI

MEMOIRE

Présenté par :

Ababacar Ousmane NIANG

Pour l'obtention du diplôme d'Ingénierie
Master Professionnel : Réseaux et Systèmes informatiques
Parcours : Informatique

Structure d'accueil : SIMEN - Système Intégré de Management de l'Éducation

Sujet :

**Etude et Mise en place d'une solution d'authentification
unique(SSO) avec des fédérations des identités**

Membres du Jury

Statut	NOM et Prénom	Grade
Président		Professeur
Superviseur de mémoire	LO Massamba	Ingénieur
Directeur de mémoire	NDIAYE Latyrs	Docteur
Examineur 1 :		
Examineur 2 :		

Présenté par : Ababacar Ousmane NIANG

Soutenu à Dakar, le 00/12/2026

Année académique : 2025-2026

DÉDICACES

Je dédie ce travail à ma famille, à mon Boss M. Sarr, à ma Petite Amie pour leurs soutiens indéfectible tout au long de mon parcours académique...

REMERCIEMENTS

Mes remerciements s'adressent d'abord à mon encadreur, M. Massamba LO, pour sa disponibilité et la qualité de son accompagnement ; au corps professoral de l'ISI ; à l'équipe du SIMEN pour son accueil durant mon stage ; ainsi qu'à tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce mémoire.

AVANT-PROPOS

L'Institut Supérieur d'Informatique (ISI) est un établissement d'enseignement supérieur agréé par l'État du Sénégal, s'imposant comme une référence dans le domaine des Technologies de l'Information et de la Communication (TIC). Il a pour mission de former de jeunes cadres opérationnels de haut niveau dans des filières professionnalisées. L'obtention du Master Professionnel au sein de l'ISI est couronnée par la réalisation d'un projet de fin de cycle au terme duquel l'étudiant est appelé à présenter le travail effectué. Ce document s'inscrit dans cette perspective, avec pour thème : « Étude et Mise en place d'une solution d'authentification unique (SSO) avec fédération d'identités ». Ce choix se justifie par le besoin croissant de sécurisation et de centralisation des identités numériques dans les organisations.

Table des matières

INTRODUCTION GÉNÉRALE.....	14
CHAPITRE I : PRÉSENTATION DU SUJET.....	16
1. Rappel sur les Systèmes d'Information et la Sécurité.....	16
1.1. Concepts fondamentaux des Systèmes d'Information (SI).....	16
1.2. Sécurité des Systèmes d'Information (SSI) et Analyse des risques.....	16
1.3. Services de sécurité et vecteurs d'attaques.....	17
1.4. Mécanismes de défense.....	17
1.5. L'Identité Numérique et le Modèle AAA.....	17
2. Contexte.....	18
2.1. Contexte général.....	18
2.2. Cadre du stage : le SIMEN.....	18
3. Problématique.....	19
4. Objectifs du mémoire.....	20
4.1. Objectif général.....	20
4.2. Objectifs spécifiques.....	20
5. Intérêts du sujet.....	21
Conclusion.....	21
CHAPITRE II : ÉTUDE DES SOLUTIONS SIMILAIRES.....	22
1. Critères de comparaison des services.....	22
2. Autres critères de comparaison.....	23
3. Étude détaillée des solutions similaires.....	23
3.1. Okta.....	23
3.2. Authentik.....	24
3.3. Zitadel.....	24
3.4. Keycloak.....	25
4. Tableau comparatif.....	26
5. Conclusion tirée du tableau comparatif.....	28
CHAPITRE III : ANALYSE DES BESOINS ET CHOIX DE LA SOLUTION TECHNIQUE	29

1. Architecture de l'existant.....	29
1.1. Description générale.....	29
1.2. Fonctionnement actuel de l'authentification.....	29
1.3. Critique de l'existant.....	30
2. Spécification des besoins.....	30
2.1. Besoins fonctionnels.....	30
2.2. Besoins non fonctionnels.....	31
CHAPITRE IV : CONCEPTION ET ARCHITECTURE DE LA SOLUTION CIBLE.....	31
1. Conception de la solution proposée.....	31
1.1. Architecture conceptuelle en couches.....	31
1.2. Cinématique d'authentification (flux Open ID Connect).....	33
1.3. Architecture technique de la maquette.....	34
1.4. Plan d'adressage et nommage.....	35
1.5. Choix techniques structurants.....	36
CHAPITRE V : MISE EN ŒUVRE PRATIQUE ET CONFIGURATION DU SERVEUR SSO.....	37
1. Mise en place de l'environnement virtualisé.....	37
1.1. Création de la machine virtuelle Ubuntu Server.....	37
1.2. Création de la machine virtuelle Windows Server.....	38
2. Mise en place du contrôleur de domaine Active Directory.....	39
2.1. Configuration réseau statique.....	39
2.2. Installation du rôle AD DS.....	39
2.3. Création de l'unité d'organisation et des utilisateurs de test.....	40
3. Test de l'interconnexion réseau.....	41
4. Expérimentation préliminaire de Keycloak en mode standalone.....	42
5. Installation de la plateforme Docker.....	42
5.1. Ajout du dépôt officiel Docker.....	42
5.2. Installation du moteur Docker.....	43
5.3. Vérification du service.....	43

5.4. Création du réseau dédié à la plateforme SSO.....	44
6. Déploiement du serveur SSO Keycloak.....	44
6.1. Arborescence du projet.....	44
6.2. Variables d'environnement.....	44
6.3. Génération du certificat TLS.....	45
6.4. Fichier de déploiement Docker Compose.....	46
6.5. Démarrage de la pile.....	46
6.6. Accès à la console et durcissement du compte d'administration.....	47
6.7. Création du realm dédié.....	48
7. Fédération des identités avec l'Active Directory.....	49
7.1. Configuration de la connexion LDAP.....	49
7.2. Périmètre et attributs de recherche.....	50
7.3. Validation de la synchronisation.....	50
CHAPITRE VI : TESTS DE VALIDATION, SÉCURITÉ (MFA) ET RÉSULTATS.....	51
1. Intégration de la première application cliente : Grafana.....	51
1.1. Déclaration du client dans Keycloak.....	51
1.2. Déploiement et configuration de Grafana.....	52
1.3. Validation de l'authentification SSO sur Grafana.....	53
2. Intégration de la seconde application cliente : WordPress.....	55
2.1. Déploiement de WordPress.....	55
2.2. Déclaration du client dans Keycloak.....	58
2.3. Installation et configuration du connecteur OIDC.....	58
2.4. Validation.....	60
3. Vue d'ensemble de la plateforme déployée.....	61
4. Sécurisation renforcée : authentification multifacteur (MFA).....	61
5. Synthèse des tests de validation.....	63
CONCLUSION GÉNÉRALE.....	64
LISTE DES RÉFÉRENCES (<i>Aux normes APA 7</i>).....	66
ANNEXES (structure proposée).....	67

LISTE DES SIGLES ET ABRÉVIATIONS

Sigle	Signification
AD / AD DS	Active Directory / Active Directory Domain Services
CNCF	Cloud Native Computing Foundation
DN	Distinguished Name
IAM	Identity and Access Management
IdP	Identity Provider (Fournisseur d'identité)
JWT	JSON Web Token
LDAP	Lightweight Directory Access Protocol
MFA	Multi-Factor Authentication
OIDC	OpenID Connect
OTP / TOTP	(Time-based) One-Time Password
OU	Organizational Unit (Unité d'organisation)
SAML	Security Assertion Markup Language
SIMEN	Système Intégré de Management de l'Éducation Nationale
SLO	Single Logout

Sigle	Signification
SP	Service Provider (Fournisseur de services)
SSO	Single Sign-On (Authentification unique)
TLS	Transport Layer Security
VM	Virtual Machine (Machine virtuelle)

LISTE DES FIGURES ET TABLEAUX

Figure 1 : Architecture conceptuelle de la solution IAM/SSO avec Keycloak

Figure 2 : Diagramme de séquence du processus d'authentification SSO (flux Authorization Code OIDC)

Figure 3 : Architecture technique de la maquette déployée

Figure 4 : Caractéristiques de la machine virtuelle Ubuntu Server sous VMware

Figure 5 : Configuration du profil utilisateur lors de l'installation d'Ubuntu Server

Figure 6 : Caractéristiques de la machine virtuelle Windows Server

Figure 7 : Configuration réseau statique du futur contrôleur de domaine

Figure 8 : Installation du rôle Active Directory Domain Services sur SRV-AD

Figure 9 : Ouverture de session sur le domaine SSO après promotion du contrôleur

Figure 10 : Unité d'organisation SSO_Utilisateurs du domaine sso.local

Figure 11 : Test de connectivité entre le serveur Ubuntu et le contrôleur de domaine (0 % de perte)

Figure 12 : Vérification de l'environnement Java (OpenJDK 21)

Figure 13 : Premier démarrage de Keycloak en mode développement

Figure 14 : Ajout de la clé GPG officielle de Docker

Figure 15 : Déclaration du dépôt Docker et mise à jour des sources APT

Figure 16 : Installation de Docker Engine et de ses composants

Figure 17 : Vérification du service Docker (état actif)

Figure 18 : Création du réseau Docker dédié « reseau-sso »

Figure 19 : Création de l'arborescence du projet Keycloak

Figure 20 : Création du fichier des variables d'environnement

Figure 21 : Contenu du fichier .env (les valeurs sensibles ont été volontairement masquées conformément aux bonnes pratiques de sécurité)

Figure 22 : Génération du certificat TLS auto-signé avec OpenSSL (RSA 4096 bits)

Figure 23 : Attribution des droits de lecture sur les certificats TLS

Figure 24 : Fichier docker-compose.yml de déploiement de Keycloak et PostgreSQL

Figure 25 : Démarrage réussi des conteneurs Keycloak et PostgreSQL

Figure 26 : Page d'authentification de la console Keycloak (HTTPS, port 8443)

Figure 27 : Création du compte administrateur permanent adm_isi

Figure 28 : Définition du mot de passe permanent (option « Temporary » désactivée)

Figure 29 : Attribution du rôle admin au compte adm_isi

Figure 30 : Création du realm dédié « Bienvenue »

Figure 31 : Paramètres de connexion de Keycloak à l'annuaire Active Directory

Figure 32 : Configuration de la recherche des utilisateurs dans l'annuaire (mode lecture seule)

Figure 33 : Synchronisation réussie des utilisateurs de l'Active Directory vers Keycloak

Figure 34 : Création du client app-grafana de type OpenID Connect

Figure 35 : Configuration des URI de redirection du client app-grafana

Figure 36 : Récupération du secret client généré par Keycloak (valeur volontairement masquée)

Figure 37 : Création de l'arborescence du projet Grafana

Figure 38 : Fichier docker-compose.yml de Grafana avec configuration OAuth2/OIDC vers Keycloak

Figure 39 : Démarrage réussi du conteneur Grafana

Figure 40 : Page d'authentification de Grafana avec l'option de connexion SSO via Keycloak

Figure 41 : Connexion SSO réussie — session Grafana de l'utilisateur Bara Ndiaye (compte fédéré depuis l'Active Directory)

Figure 42 : Création de l'arborescence du projet WordPress

Figure 43 : Fichier docker-compose.yml de WordPress et de sa base de données MariaDB

Figure 44 : Démarrage réussi des conteneurs WordPress et MariaDB

Figure 45 : Lancement de l'assistant d'installation de WordPress

Figure 46 : Configuration du site ISI-SSO et du compte administrateur (valeurs sensibles masquées)

Figure 47 : Confirmation de l'installation de WordPress

Figure 48 : Accès au tableau de bord d'administration de WordPress

Figure 49 : Création du client app-wordpress de type OpenID Connect

Figure 50 : Installation du plugin OpenID Connect Generic Client

Figure 51 : Configuration du plugin OIDC avec les points de terminaison Keycloak (secret masqué)

Figure 52 : Page d'authentification WordPress avec l'option de connexion SSO

Figure 53 : Connexion SSO réussie sur WordPress avec un compte fédéré depuis l'Active Directory

Figure 54 : Vue d'ensemble des conteneurs de la plateforme SSO en fonctionnement

Figure 55 : Enrôlement de l'authentification forte — configuration de l'application mobile OTP

Tableau comparatif : Critères / Solutions

Tableau 1 : Récapitulatif des services conteneurisés de la plateforme

Tableau 2 : Synthèse des tests de validation de la solution

RÉSUMÉ

La multiplication des applications au sein des systèmes d'information contraint les utilisateurs à gérer de nombreux identifiants, fragilisant la sécurité et dégradant l'expérience utilisateur. Ce mémoire étudie et met en place une solution d'authentification unique (SSO) fondée sur la fédération des identités. Après une étude comparative des solutions du marché (Okta, Authentik, Zitadel, Keycloak), la solution open source Keycloak a été retenue et déployée dans un environnement conteneurisé (Docker) sécurisé par TLS. Les identités d'un annuaire Microsoft Active Directory ont été fédérées vers Keycloak, puis deux applications hétérogènes — Grafana et WordPress — ont été intégrées via le protocole OpenID Connect. La sécurité des accès a été renforcée par une authentification multifacteur (TOTP). Les tests de validation démontrent qu'une authentification unique ouvre l'accès à l'ensemble des applications, confirmant la pertinence de l'architecture proposée pour les organisations souhaitant centraliser leurs accès tout en conservant la souveraineté de leurs données.

Mots-clés : SSO, IAM, fédération d'identités, Keycloak, OpenID Connect, Active Directory, MFA, Docker.

ABSTRACT

The proliferation of applications within information systems forces users to manage numerous credentials, weakening security and degrading user experience. This thesis studies and implements a Single Sign-On (SSO) solution based on identity federation. Following a comparative study of market solutions (Okta, Authentik, Zitadel, Keycloak), the open-source solution Keycloak was selected and deployed in a TLS-secured containerized environment (Docker). Identities from a Microsoft Active Directory were federated to Keycloak, and two heterogeneous applications — Grafana and WordPress — were integrated using the OpenID Connect protocol. Access security was strengthened with multi-factor authentication (TOTP). Validation tests demonstrate that a single authentication grants access to all applications, confirming the relevance of the proposed architecture for organizations seeking centralized access management with full data sovereignty.

Keywords: SSO, IAM, identity federation, Keycloak, OpenID Connect, Active Directory, MFA, Docker.

INTRODUCTION GÉNÉRALE

À l'ère de la transformation numérique, les organisations publiques et privées voient leur patrimoine applicatif croître de manière exponentielle. Messageries collaboratives, plateformes métiers, outils de supervision et portails web se multiplient pour répondre aux exigences de modernisation des processus. Toutefois, cette expansion s'accompagne d'un défi structurel majeur : la gestion et la sécurisation des accès utilisateurs. Historiquement, chaque application a été déployée avec son propre référentiel de comptes et son mécanisme d'authentification autonome. Cette fragmentation des identités numériques contraint aujourd'hui les utilisateurs à multiplier les identifiants, fragilisant considérablement la sécurité globale des infrastructures. Les rapports récents en cybersécurité s'accordent sur un point critique : le vol ou la compromission des identifiants demeure le premier vecteur d'intrusion au sein des systèmes d'information (Microsoft, 2025).

Le Sénégal, résolument engagé dans une stratégie ambitieuse de digitalisation de son administration, n'échappe pas à cette réalité. Le secteur de l'éducation nationale, dont le système d'information est piloté par le SIMEN (Système Intégré de Management de l'Éducation Nationale), illustre parfaitement cet enjeu à grande échelle. Au quotidien, de nombreux agents accèdent à des services hétérogènes, répartis entre des serveurs locaux et des environnements cloud, chacun exigeant la gestion de ses propres codes d'accès. Cette dispersion engendre une surcharge cognitive pour les utilisateurs — qui adoptent par conséquent des pratiques à risque telles que la réutilisation de mots de passe ou leur consignation sur des supports non sécurisés —

tout en alourdissant la charge des équipes de support technique et en élargissant la surface d'attaque de l'organisation.

Face à ces vulnérabilités, la discipline de la Gestion des Identités et des Accès (IAM — *Identity and Access Management*) apporte une réponse architecturale incontournable. L'authentification unique (SSO — *Single Sign-On*) et la fédération d'identités en constituent les piliers fondamentaux. Elles permettent à un utilisateur de s'authentifier une unique auprès d'un fournisseur centralisé, puis de propager cette confiance de manière sécurisée et transparente vers l'ensemble du parc applicatif, en s'appuyant sur des protocoles ouverts et standardisés tels que SAML 2.0, OAuth 2.0 et OpenID Connect (OIDC).

C'est dans ce contexte que s'inscrit ce travail de fin d'études. Ce mémoire a pour objet l'étude et la mise en place d'une solution d'authentification unique souveraine et sécurisée, adossée à un annuaire d'entreprise Microsoft Active Directory et intégrée à des services conteneurisés via Docker. L'objectif est de démontrer la faisabilité et la robustesse d'une telle architecture dans un environnement de laboratoire représentatif des besoins réels d'une grande structure comme le SIMEN.

Afin de structurer notre démarche et d'apporter une réponse rigoureuse à la problématique soulevée, ce mémoire s'articule autour de six chapitres principaux :

- **Le Chapitre I : État de l'art sur la gestion des identités et des accès (IAM)**, qui pose le contexte théorique, analyse les concepts de la sécurité des systèmes d'information et introduit la problématique de la fragmentation des identités.
- **Le Chapitre II : Étude théorique du SSO et de la fédération d'identités**, qui réalise une étude comparative approfondie des solutions du marché (Okta, Authentik, Zitadel, Keycloak) afin de justifier le choix technologique retenu.
- **Le Chapitre III : Analyse des besoins et choix de la solution technique**, qui formalise les exigences fonctionnelles et non fonctionnelles de la plateforme et détaille la conception architecturale de la solution cible (modèle en couches et cinématique OIDC).
- **Le Chapitre IV : Conception et architecture de la solution cible**, qui prépare le terrain organisationnel et technique de la maquette virtualisée.
- **Le Chapitre V : Mise en œuvre pratique et configuration du serveur SSO**, qui expose pas à pas le déploiement de l'environnement sous Ubuntu Server et Windows

Server, la conteneurisation Docker, la fédération de l'annuaire LDAP et l'intégration des applications clientes (Grafana et WordPress).

- **Le Chapitre VI : Tests de validation, sécurité (MFA) et résultats**, qui présente la sécurisation par authentification multifacteur (TOTP) ainsi que la synthèse des tests fonctionnels valant validation de la plateforme.

CHAPITRE I : PRÉSENTATION DU SUJET

Introduction

La transformation numérique des administrations publiques sénégalaises s'est considérablement accélérée au cours de la dernière décennie, portée par les stratégies nationales de digitalisation. Le secteur de l'éducation nationale, qui compte parmi les plus vastes réseaux administratifs du pays, n'échappe pas à cette dynamique : gestion des personnels, suivi des élèves, plateformes pédagogiques, messageries professionnelles et outils de pilotage se sont multipliés au sein du ministère. Cette croissance du parc applicatif, si elle témoigne d'une modernisation réelle, s'accompagne néanmoins d'un défi majeur : la gestion des identités et des accès des utilisateurs.

Le présent chapitre pose les fondations de ce mémoire. Il présente d'abord le contexte général et la structure d'accueil, expose ensuite la problématique qui motive ce travail, définit les objectifs poursuivis et justifie enfin l'intérêt du sujet sur les plans scientifique, organisationnel et sécuritaire.

1. Rappel sur les Systèmes d'Information et la Sécurité

1.1. Concepts fondamentaux des Systèmes d'Information (SI)

Dans un contexte de transformation numérique accélérée, le Système d'Information (SI) constitue le centre névralgique de toute organisation. Un SI se définit comme un ensemble

organisé de ressources matérielles, logicielles, humaines, de données et de procédures permettant d'acquérir, de traiter, de stocker et de communiquer des informations. Sa vocation est d'aligner la gestion des flux d'informations sur les objectifs stratégiques et opérationnels de l'organisation. L'ouverture croissante de ces systèmes a cependant fait exploser leur complexité et, par conséquent, les défis liés à leur sécurisation.

1.2. Sécurité des Systèmes d'Information (SSI) et Analyse des risques

La Sécurité des Systèmes d'Information (SSI) vise à garantir que les ressources du SI sont protégées contre toute compromission. Pour appréhender cette discipline, il est indispensable de définir :

- **Les Actifs** : Toute ressource (donnée, serveur, application, infrastructure) ayant de la valeur pour l'organisation et nécessitant une protection.
- **Les Vulnérabilités** : Les failles ou faiblesses inhérentes à un actif (défaut de configuration, faille logicielle, mauvaise pratique humaine) susceptibles d'être exploitées.
- **Les Menaces** : Les causes potentielles d'un incident, intentionnelles (piratage) ou accidentelles (panne), pouvant exploiter une vulnérabilité.
- **La Surface d'attaque** : L'ensemble des points d'entrée exposés par le SI qu'un attaquant pourrait exploiter.

1.3. Services de sécurité et vecteurs d'attaques

Pour protéger efficacement un système, la sécurité s'articule autour des services fondamentaux (DICT) :

- **La Confidentialité** : Garantit que l'information n'est accessible qu'aux entités autorisées (Attaques : interception de trafic, vol d'identifiants).
- **L'Intégrité** : Assure que les données n'ont pas été altérées (Attaques : falsification, injection).
- **La Disponibilité** : Maintient l'accès aux services dans les délais attendus (Attaques : Déni de service DoS/DDoS, ransomwares).
- **La Traçabilité (Accounting)** : Permet de conserver les traces des actions effectuées sur le système.

1.4. Mécanismes de défense

Face à ces attaques, les organisations déploient diverses contre-mesures. Historiquement périmétrique (pare-feux, IDS/IPS, VPN), la défense s'efface aujourd'hui au profit d'un nouveau paradigme : le périmètre identitaire, rendant cruciale la gestion des accès.

1.5. L'Identité Numérique et le Modèle AAA

La sécurité des accès modernes repose sur la définition stricte de l'identité numérique, le critère unique et univoque permettant de reconnaître un individu au sein d'un système. L'architecture de gestion de cette identité s'appuie sur le modèle AAA :

- **L'Authentification (Authentication)** : Processus par lequel le système vérifie que l'entité est bien celle qu'elle prétend être, en s'appuyant sur ce que l'utilisateur connaît (mot de passe), possède (téléphone, token) ou est (biométrie).
- **L'Autorisation (Authorization)** : Étape déterminant les droits et les privilèges de l'entité. Elle vérifie à quelles ressources l'utilisateur a le droit d'accéder.
- **La Traçabilité (Accounting)** : Enregistrement continu des actions réalisées par l'utilisateur authentifié.

2. Contexte

2.1. Contexte général

Avec l'accélération de la transformation digitale au Sénégal, les organisations — banques, administrations publiques, PME — exploitent un nombre croissant d'applications métiers, souvent réparties entre des serveurs locaux et des services hébergés dans le Cloud. Chaque application embarque historiquement son propre mécanisme d'authentification : un utilisateur doit ainsi créer et mémoriser autant de couples identifiant/mot de passe qu'il utilise de services.

Cette fragmentation des identités numériques engendre des conséquences bien documentées : affaiblissement des mots de passe (réutilisation, simplification), pratiques à risque (mots de passe notés sur des supports non sécurisés), multiplication des demandes de réinitialisation auprès des équipes de support, et surtout élargissement de la surface d'attaque exploitable par des acteurs malveillants. Dans un contexte où les cyberattaques ciblant les administrations africaines sont en constante augmentation, la centralisation et la sécurisation de la gestion des identités deviennent un enjeu stratégique.

Les grandes organisations technologiques mondiales ont depuis longtemps généralisé l'authentification unique : un compte Google ouvre l'accès à Gmail, Drive, YouTube et des milliers de services tiers ; un compte Microsoft fédère Office 365, Teams et Azure. Ces modèles reposent sur des standards ouverts de fédération d'identités — SAML 2.0, OAuth 2.0, OpenID Connect — aujourd'hui matures et accessibles à toute organisation souhaitant les mettre en œuvre sur sa propre infrastructure.

2.2. Cadre du stage : le SIMEN

Le présent travail a été réalisé dans le cadre d'un stage au sein du **SIMEN (Système Intégré de Management de l'Éducation Nationale)**, entité rattachée au Ministère de l'Éducation nationale du Sénégal et chargée du pilotage du système d'information du secteur éducatif. Le SIMEN a pour vocation d'unifier et de moderniser la gestion numérique de l'éducation nationale : administration des infrastructures et des services numériques, développement et exploitation des applications de gestion (personnels, élèves, établissements), et accompagnement des utilisateurs à l'échelle nationale.

De par sa mission transversale, le SIMEN administre un portefeuille applicatif hétérogène servant un très grand nombre d'agents répartis sur l'ensemble du territoire. Cette position en fait un terrain particulièrement pertinent pour l'étude et la mise en place d'une solution d'authentification unique adossée à une fédération d'identités : les problématiques de multiplication des comptes, de sécurité des accès et de charge du support s'y expriment à grande échelle.

C'est dans ce cadre que s'inscrit ce mémoire : concevoir et démontrer, sur une maquette représentative de l'environnement technique du SIMEN, une architecture SSO complète, souveraine et reproductible.

3. Problématique

La multiplication des applications au sein du système d'information oblige chaque utilisateur à gérer de nombreux identifiants. Cette situation engendre trois catégories de problèmes.

Sur le plan de la sécurité : la surcharge cognitive pousse les utilisateurs à adopter des mots de passe faibles ou identiques sur plusieurs services, voire à les consigner sur des supports non sécurisés. En cas de compromission d'un seul compte, un attaquant peut rebondir sur l'ensemble des services partageant le même secret. Par ailleurs, la dispersion des bases de comptes rend

impossible toute politique de sécurité homogène : désactiver l'ensemble des accès d'un agent quittant l'organisation devient une opération longue et sujette aux oublis.

Sur le plan opérationnel : les demandes de réinitialisation de mots de passe représentent une part significative des sollicitations du support technique, mobilisant des ressources humaines au détriment de tâches à plus forte valeur ajoutée.

Sur le plan de l'expérience utilisateur : les authentifications répétées fragmentent les parcours de travail et réduisent la productivité des agents.

Dès lors, la question centrale de ce mémoire peut être formulée ainsi :

Comment centraliser l'authentification et l'accès à des ressources hétérogènes (locales et cloud) tout en garantissant un niveau de sécurité élevé et une expérience utilisateur fluide ?

De cette question principale découlent trois questions secondaires :

1. Quels protocoles de fédération d'identités (SAML 2.0, OAuth 2.0, OpenID Connect) répondent le mieux aux besoins d'interopérabilité du système d'information ?
2. Comment fédérer un annuaire d'entreprise existant (Microsoft Active Directory) avec un fournisseur d'identité centralisé ?
3. Comment renforcer l'authentification par un second facteur (MFA) sans dégrader l'expérience utilisateur ?

4. Objectifs du mémoire

4.1. Objectif général

Étudier et mettre en place une solution d'authentification unique (SSO) basée sur la fédération d'identités, permettant de centraliser et de sécuriser l'accès aux applications du système d'information.

4.2. Objectifs spécifiques

- **OS1** : Étudier les concepts fondamentaux de la gestion des identités et des accès (IAM) ainsi que les protocoles de fédération (SAML 2.0, OAuth 2.0, OpenID Connect) ;
- **OS2** : Comparer les principales solutions SSO du marché et justifier le choix d'une solution open source souveraine ;

- **OS3** : Déployer un fournisseur d'identité centralisé (Keycloak) dans un environnement conteneurisé sécurisé (Docker, TLS) ;
- **OS4** : Fédérer les identités d'un annuaire Microsoft Active Directory vers le fournisseur d'identité ;
- **OS5** : Intégrer des applications clientes hétérogènes (Grafana, WordPress) via le protocole OpenID Connect ;
- **OS6** : Renforcer la sécurité des accès par une authentification multifacteur (MFA/OTP) et valider l'ensemble par des tests fonctionnels.

5. Intérêts du sujet

- **Souveraineté et sécurité des données.** En s'appuyant sur Keycloak, solution open source de référence, l'organisation garde le contrôle total de ses identités sans dépendre exclusivement des géants du Cloud, répondant ainsi aux exigences de protection des données sensibles au Sénégal. Les identités des agents de l'État demeurent hébergées sur l'infrastructure nationale.
- **Renforcement de la posture de cybersécurité :** L'implémentation du MFA (authentification forte) et de la fédération d'identités permet de bloquer efficacement les tentatives d'accès non autorisés, même en cas de vol de mot de passe. La centralisation offre en outre un point unique d'audit et de traçabilité des connexions.
- **Efficacité opérationnelle :** La solution réduit drastiquement les coûts liés au support technique (perte de mots de passe) et améliore immédiatement la productivité des collaborateurs, qui n'ont plus qu'à s'authentifier une seule fois pour accéder à l'ensemble de leurs applications.
- **Intérêt académique et reproductibilité :** Ce travail documente de bout en bout — de la virtualisation à l'intégration applicative — une architecture SSO complète fondée sur des standards ouverts. Il constitue ainsi une référence pratique francophone réutilisable par d'autres administrations ou entreprises sénégalaises souhaitant engager une démarche similaire.

Conclusion

Ce chapitre a présenté le cadre général du sujet, la structure d'accueil, la problématique de la fragmentation des identités numériques ainsi que les objectifs et intérêts du travail. La question centrale étant posée — centraliser les accès sans compromettre ni la sécurité ni l'expérience utilisateur —, il convient à présent d'examiner les solutions existantes sur le marché afin de fonder un choix technique rigoureux. C'est l'objet du chapitre suivant.

CHAPITRE II : ÉTUDE DES SOLUTIONS SIMILAIRES

Introduction

Avant de retenir une solution technique, il est indispensable de recenser et d'évaluer les offres existantes sur le marché de la gestion des identités et des accès. Le paysage des solutions SSO se divise en deux grandes familles : les plateformes propriétaires hébergées dans le Cloud (Identity as a Service - IDaaS) et les solutions open source déployables sur l'infrastructure de l'organisation (on-premise). Ce chapitre définit d'abord les critères de comparaison retenus, étudie ensuite en détail quatre solutions représentatives - Okta, Authentik, Zitadel et Keycloak puis synthétise cette analyse dans un tableau comparatif dont découle le choix final.

1. Critères de comparaison des services

Six critères principaux ont été retenus, en cohérence avec les besoins exprimés dans la problématique :

- **Modèle et licence** : solution propriétaire ou open source, mode de déploiement (Cloud, on-premise, hybride). Ce critère conditionne la maîtrise de la solution et l'indépendance vis-à-vis d'un éditeur ;

- **Coût financier** : coût des licences et d'exploitation, critère déterminant dans le contexte d'une administration publique ;
- **Support des protocoles OIDC/SAML** : richesse de l'implémentation des standards de fédération (Open ID Connect, OAuth 2.0, SAML 2.0), garantie d'interopérabilité avec des applications hétérogènes ;
- **Intégration Active Directory** : capacité à fédérer un annuaire d'entreprise existant, besoin central du présent projet ;
- **Simplicité de déploiement** : effort d'installation, de configuration et de maintenance ;
- **Souveraineté des données** : localisation et contrôle des données d'identité, enjeu stratégique pour les données sensibles de l'État sénégalais.

2. Autres critères de comparaison

À ces critères principaux s'ajoutent des critères secondaires qui affinent l'évaluation :

- La **maturité du projet** et la taille de sa communauté, gages de pérennité et de réactivité en cas de faille de sécurité ;
- La **qualité de la documentation** officielle et l'abondance des retours d'expérience ;
- Le **support natif du MFA** (authentification multi facteur) ;
- La **scalabilité** : capacité de montée en charge et de déploiement en haute disponibilité ;
- La **compatibilité avec la conteneurisation** : disponibilité d'images Docker officielles maintenues.

3. Étude détaillée des solutions similaires

3.1. Okta

Okta est le leader mondial du marché IDaaS. Fondée en 2009, la société propose une plateforme cloud complète intégrant SSO, MFA adaptatif, gestion du cycle de vie des identités et un catalogue de plusieurs milliers d'intégrations pré-configurées.

Avantages :

- Plateforme extrêmement mature et riche fonctionnellement ;
- Déploiement immédiat, aucune infrastructure à gérer ;
- Excellent support des protocoles OIDC et SAML ;
- Intégration Active Directory éprouvée via l'agent Okta AD.

Inconvénients :

- Coût très élevé, facturé par utilisateur et par mois, prohibitif à l'échelle d'une organisation comptant des milliers d'agents ;
- Données d'identité hébergées dans le cloud public de l'éditeur, hors du territoire national : souveraineté très faible ;
- Dépendance forte envers un fournisseur étranger (vendor lock-in) ;
- Nécessite une connectivité Internet permanente.

3.2. Authentik

Authentik est un fournisseur d'identité open source moderne, apparu en 2020, qui se distingue par une interface d'administration soignée et une approche orientée simplicité. Il se déploie principalement via Docker et vise les PME et les infrastructures auto-hébergées.

Avantages :

- Open source et gratuit (édition communautaire) ;
- Interface intuitive, prise en main rapide ;
- Déploiement Docker natif, facile à mettre en œuvre ;
- Bon support OIDC et SAML, avec des fonctionnalités de proxy d'authentification pour les applications anciennes ;
- Souveraineté totale grâce à l'hébergement local.

Inconvénients :

- Projet jeune : communauté et écosystème plus restreints ;
- Intégration Active Directory possible mais nécessitant une configuration avancée ;

- Peu de retours d'expérience en production à grande échelle ;
- Documentation moins exhaustive que celle des solutions établies.

3.3. Zitadel

Zitadel est une solution IAM open source d'origine suisse, conçue nativement pour le cloud et écrite en Go. Elle met l'accent sur l'architecture multi-tenant et l'event sourcing, garantissant une traçabilité complète des événements d'identité.

Avantages :

- Open source, disponible en auto-hébergèrent ou en SaaS managé, au choix ;
- Architecture moderne, performante et nativement scalable ;
- Excellente traçabilité des événements (audit par event sourcing) ;
- Bon support OIDC (implémentation certifiée Open ID).

Inconvénients :

- Support SAML plus limité que celui de ses concurrents ;
- Intégration Active Directory moyenne, moins aboutie ;
- Communauté encore modeste, documentation francophone quasi inexistante ;
- Complexité de déploiement modérée (dépendances de base de données spécifiques).

3.4. Keycloak

Keycloak est la solution open source de référence en matière de gestion des identités et des accès. Initié en 2014 et soutenu par Red Hat — il constitue la base du produit commercial Red Hat build of Keycloak —, le projet a rejoint en 2023 la **Cloud Native Computing Foundation (CNCF)**, gage de pérennité et de gouvernance ouverte. Keycloak fournit nativement : SSO, fédération d'annuaires LDAP/Active Directory, brokering d'identités externes, MFA/OTP, gestion fine des rôles et des sessions, et thèmes personnalisables.

Avantages :

- Open source et totalement gratuit, soutenu par Red Hat et la CNCF ;
- Support excellent et certifié des protocoles OIDC, OAuth 2.0 et SAML 2.0 ;

- **Fédération Active Directory/LDAP native et bidirectionnelle**, sans agent tiers ;
- MFA/OTP intégré en standard ;
- Très vaste communauté, documentation abondante, nombreux retours de production ;
- Image Docker officielle, parfaitement adaptée à la conteneurisation ;
- Souveraineté totale des données grâce à l'hébergement on-premise.

Inconvénients :

- Courbe d'apprentissage non négligeable, liée à la richesse fonctionnelle ;
- Consommation mémoire relativement élevée (machine virtuelle Java) ;
- Interface d'administration dense pour un utilisateur débutant ;
- Déploiement de production exigeant (base de données externe, configuration TLS).

4. Tableau comparatif

Le tableau suivant synthétise l'évaluation des quatre solutions au regard des critères principaux.

TABLEAU COMPARATIF

Critères / Solutions	Okta	Authentik	Zitadel	Keycloak
Modèle / Licence	Propriétaire (Cloud)	Open-Source (On-Premise)	Open-Source (On-Premise / Cloud)	Open-Source (On-Premise)
Coût financier	Très élevé (par utilisateur)	Gratuit	Gratuit	Gratuit
Support OIDC / SAML	Excellent	Bon	Bon (SAML plus limité)	Excellent
Intégration Active Directory	Excellente (via agent Okta)	Bonne (configuration avancée)	Moyenne	Excellente (native et bidirectionnelle)
Simplicité de déploiement	Très élevée (Cloud)	Facile	Modérée	Modérée
Souveraineté des données	Faible (Cloud public)	Totale (hébergement local)	Totale (ou Cloud selon choix)	Totale (hébergement local)

5. Conclusion tirée du tableau comparatif

L'analyse croisée du tableau fait ressortir les enseignements suivants :

- **Okta** est écarté malgré sa maturité : son modèle de facturation par utilisateur est incompatible avec le budget d'une administration publique, et l'hébergement des identités des agents de l'État dans un cloud étranger contrevient à l'exigence de souveraineté ;
- **Authentik** séduit par sa simplicité de déploiement, mais sa relative jeunesse et son intégration Active Directory moins aboutie constituent des risques pour un déploiement institutionnel ;
- **Zitadel** propose une architecture moderne, mais son support SAML limité et son intégration AD moyenne le pénalisent face au besoin central de fédération d'annuaire ;
- **Keycloak** cumule les atouts décisifs : gratuité, support excellent et certifié des trois protocoles de fédération, **intégration Active Directory native et bidirectionnelle** — cœur du présent projet —, MFA intégré, souveraineté totale et écosystème le plus mature du marché open source.

Keycloak est donc retenu comme solution SSO pour la suite de ce travail. Sa complexité de déploiement, seul point de vigilance identifié, sera maîtrisée par la conteneurisation Docker présentée au CHAPITRE IV.

Conclusion

Ce chapitre a passé en revue quatre solutions représentatives du marché du SSO au regard de critères adaptés au contexte d'une administration publique sénégalaise. L'étude comparative a abouti au choix motivé de Keycloak. Le chapitre suivant analyse l'existant et conçoit l'architecture de la solution cible.

CHAPITRE III : ANALYSE DES BESOINS ET CHOIX DE LA SOLUTION TECHNIQUE

Introduction

Le choix de Keycloak étant arrêté, il convient à présent d'analyser l'environnement dans lequel la solution s'inscrira, puis de concevoir l'architecture cible. Ce chapitre examine d'abord l'architecture de l'existant et ses limites du point de vue de la gestion des identités. Il spécifie ensuite les besoins fonctionnels et non fonctionnels de la solution, avant de présenter la conception proprement dite : architecture conceptuelle en couches, cinématique d'authentification et architecture technique de la maquette qui sera réalisée au chapitre suivant.

1. Architecture de l'existant

1.1. Description générale

Le système d'information étudié repose sur un modèle classique dans lequel chaque application gère sa propre base d'utilisateurs. L'environnement type se compose :

- D'un **annuaire Microsoft Active Directory**, utilisé principalement pour l'ouverture de session sur les postes de travail du domaine et le partage de ressources internes ;
- D'applications **web métiers** (portails, outils de supervision, plateformes de gestion de contenu) hébergées sur des serveurs locaux ou dans le cloud, chacune disposant de son propre référentiel de comptes et de sa propre page de connexion ;
- De **postes clients** Windows et de navigateurs web par lesquels les agents accèdent aux services.

1.2. Fonctionnement actuel de l'authentification

Dans l'architecture existante, le processus d'authentification est décentralisé : lorsqu'un agent souhaite accéder à une application, celle-ci vérifie les identifiants saisis contre sa propre base locale de comptes. L'annuaire Active Directory, bien que présent, n'est pas exploité par les applications web : il n'existe aucun lien entre l'identité « domaine » de l'agent et ses identités applicatives.

1.3. Critique de l'existant

L'analyse de ce fonctionnement met en évidence plusieurs limites majeures :

Limite	Conséquence
Multiplication des référentiels de comptes	Chaque application maintient sa propre base : incohérences, comptes orphelins, doublons
Multiplication des mots de passe	Mots de passe faibles ou réutilisés, notes sur papier, risque de compromission en chaîne
Absence de cycle de vie centralisé	Au départ d'un agent, ses comptes applicatifs ne sont pas systématiquement désactivés : failles de sécurité durables
Charge du support technique	Volume important de demandes de réinitialisation de mots de passe
Absence de traçabilité globale	Impossible d'auditer de façon centralisée « qui accède à quoi et quand »
Absence d'authentification forte	Aucun second facteur : un mot de passe volé suffit à compromettre un compte

Ce constat justifie pleinement la mise en place d'une solution centralisée de gestion des identités et des accès, fondée sur le SSO et la fédération d'identités.

2. Spécification des besoins

2.1. Besoins fonctionnels

La solution cible doit permettre :

- **BF1** : l'authentification unique (SSO) — un agent authentifié une fois accède à toutes les applications intégrées sans ressaisir ses identifiants ;
- **BF2** : la fédération de l'annuaire Active Directory existant — les comptes du domaine sont la source unique de vérité des identités ;
- **BF3** : l'intégration d'applications hétérogènes via des protocoles standards (Open ID Connect en priorité, SAML 2.0 en capacité) ;
- **BF4** : l'authentification multi facteur (MFA/OTP) pour renforcer la sécurité des accès ;

- **BF5** : la gestion centralisée des utilisateurs, des rôles et des sessions (dont la déconnexion globale — Single Logout) ;
- **BF6** : le provisioning automatique des comptes applicatifs à la première connexion fédérée.

2.2. Besoins non fonctionnels

- **BNF1 — Sécurité** : chiffrement TLS de tous les échanges ; secrets clients protégés ; annuaire accédé en lecture seule ;
- **BNF2 — Souveraineté** : hébergement intégral sur l'infrastructure de l'organisation (on-premise) ;
- **BNF3 — Disponibilité** : redémarrage automatique des services (politique de restart des conteneurs) ;
- **BNF4 — Maintenabilité** : déploiement conteneurisé, reproductible et documenté (Infrastructure as Code via Docker Compose) ;
- **BNF5 — Coût** : recours exclusif à des solutions open source gratuites ;
- **BNF6 — Expérience utilisateur** : parcours de connexion simple et uniforme sur toutes les applications.

CHAPITRE IV : CONCEPTION ET ARCHITECTURE DE LA SOLUTION CIBLE

1. Conception de la solution proposée

1.1. Architecture conceptuelle en couches

L'architecture cible s'organise en six couches complémentaires, illustrées par la figure ci-dessous.

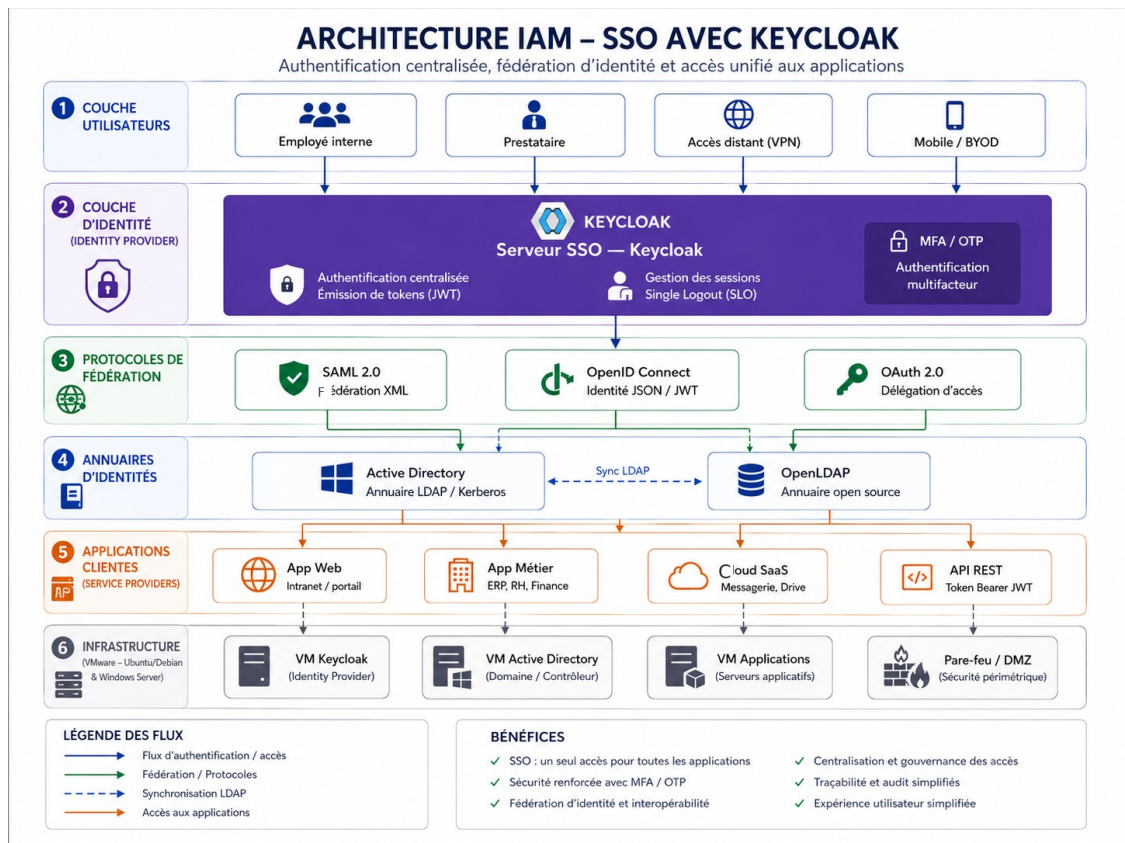


Figure 3.1 : Architecture conceptuelle de la solution IAM/SSO avec Keycloak

- **Couche 1 — Utilisateurs** : employés internes, prestataires, accès distants (VPN) et terminaux mobiles (BYOD) constituent les consommateurs des services ;
- **Couche 2 — Fournisseur d'identité (IdP)** : Keycloak centralise l'authentification, émet les jetons (JWT), gère les sessions et la déconnexion globale (SLO), et applique l'authentification multi facteur (MFA/OTP) ;
- **Couche 3 — Protocoles de fédération** : SAML 2.0, Open ID Connect et OAuth 2.0 assurent la propagation sécurisée et standardisée de l'identité vers les applications ;
- **Couche 4 — Annuaire d'identités** : l'Active Directory (et, par extension, tout annuaire OpenLDAP) demeure la source de vérité des comptes, synchronisée avec Keycloak par fédération LDAP ;
- **Couche 5 — Applications clientes (SP)** : applications web internes, applications métiers, services SaaS et API REST délèguent leur authentification à l'IdP ;

- **Couche 6 — Infrastructure** : machines virtuelles VMware (Ubuntu Server pour l'IdP et les applications, Windows Server pour le contrôleur de domaine) et sécurité périmétrique.

1.2. Cinématique d'authentification (flux Open ID Connect)

Le protocole retenu pour l'intégration des applications est **Open ID Connect** (flux Authorization Code), dont la cinématique complète est modélisée par le diagramme de séquence suivant.

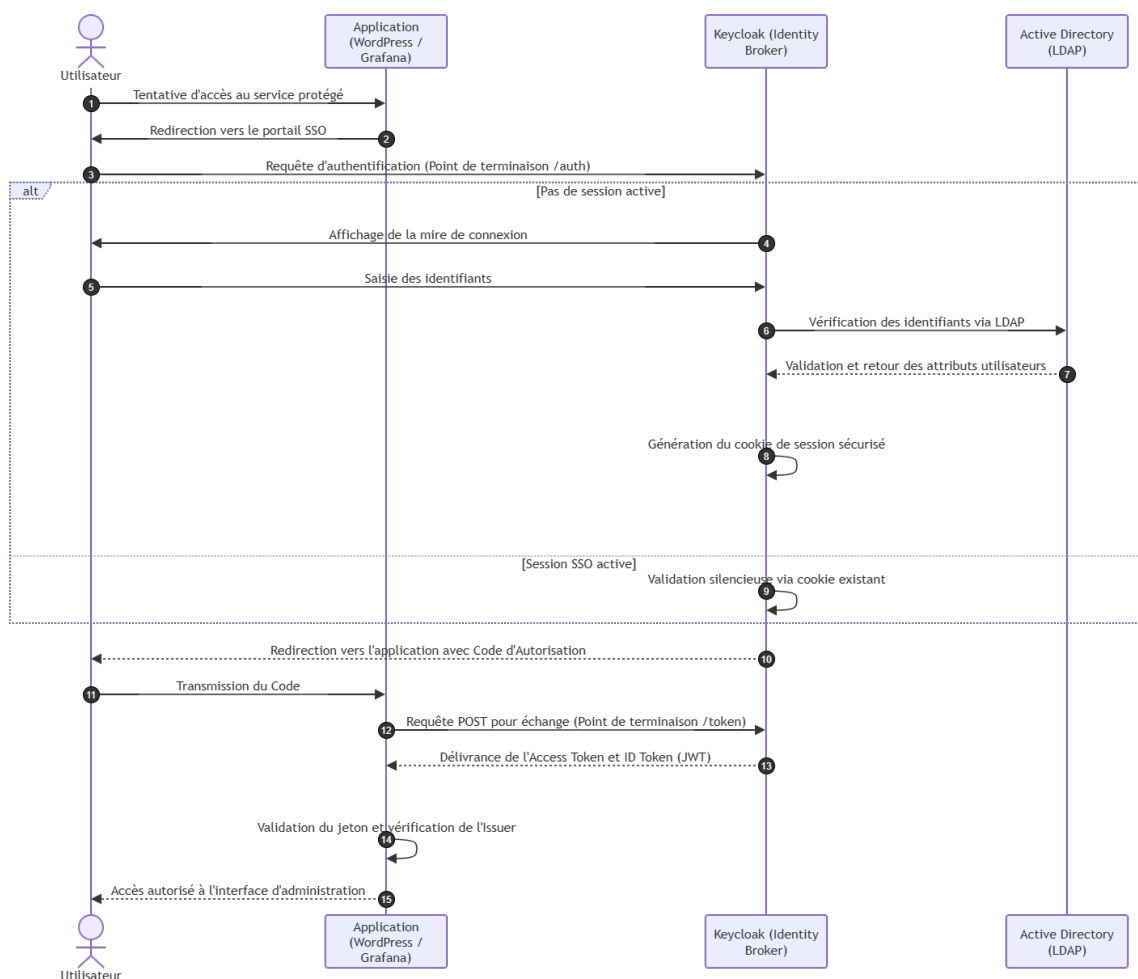


Figure 3.2 : Diagramme de séquence du processus d'authentification SSO (flux Authorization Code OIDC)

Le déroulement est le suivant :

1. L'utilisateur tente d'accéder à une application protégée (Grafana ou WordPress) ;
2. Ne détenant pas de session locale, l'application redirige le navigateur vers Keycloak (point de terminaison /auth) ;
3. **Deux cas se présentent alors :**

- **Absence de session SSO** : Keycloak affiche sa mire de connexion ; l'utilisateur saisit ses identifiants, que Keycloak vérifie auprès de l'Active Directory via LDAP ; en cas de succès, un cookie de session sécurisé est créé ;
 - **Session SSO déjà active** : Keycloak valide silencieusement la session existante grâce au cookie — **l'utilisateur n'a rien à saisir : c'est le cœur du mécanisme SSO** ;
4. Keycloak redirige l'utilisateur vers l'application avec un code d'autorisation à usage unique ;
 5. L'application échange ce code contre les jetons (Access Token et ID Token au format JWT) auprès du point de terminaison /Token ;
 6. Après validation du jeton et vérification de l'émetteur (issuer), l'application ouvre la session : l'accès est autorisé.

Ce mécanisme garantit que les identifiants de l'utilisateur ne transitent **jamais** par les applications clientes : seul Keycloak les manipule, ce qui réduit considérablement la surface d'attaque.

1.3. Architecture technique de la maquette

La solution est matérialisée par une maquette virtualisée sous VMware Workstation, représentative d'un environnement de production.

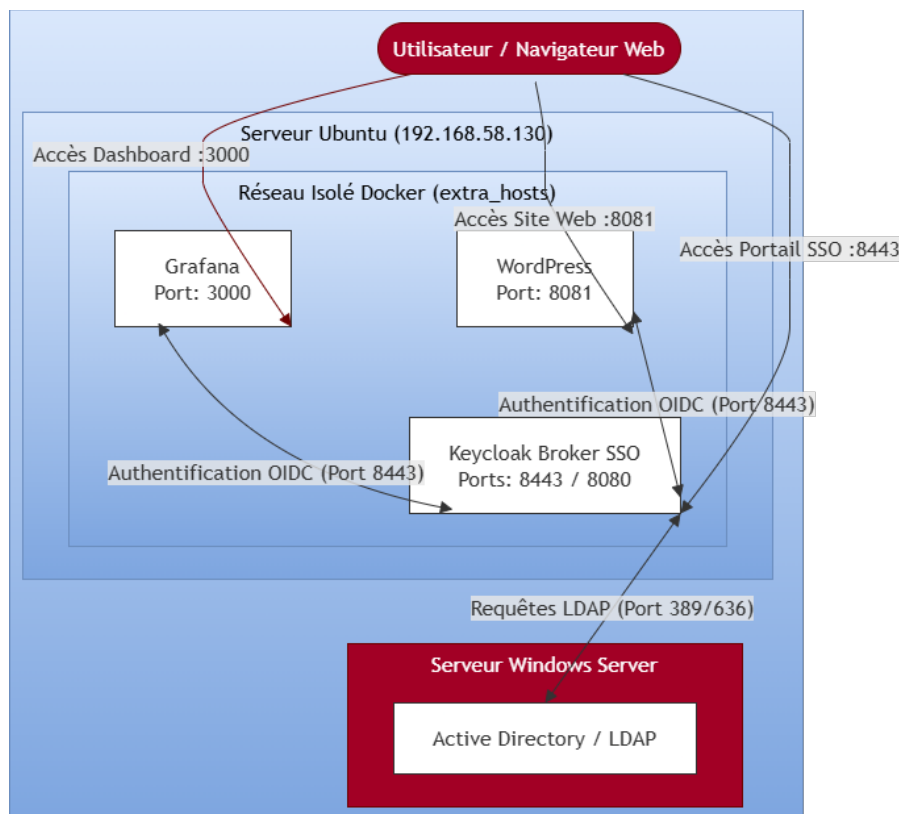


Figure 3.3 : Architecture technique de la maquette déployée

L'architecture technique comprend :

- Une **VM Ubuntu Server** (192.168.58.130) hébergeant l'écosystème conteneurisé Docker : le serveur Keycloak (ports 8080/8443) adossé à une base PostgreSQL, l'application Grafana (port 3000) et l'application WordPress (port 8081) adossée à Maria DB. Tous les conteneurs communiquent au sein d'un réseau Docker dédié et isolé (reseau-sso) ;
- Une **VM Windows Server** (192.168.10.10) jouant le rôle de contrôleur de domaine Active Directory (domaine sso.local), interrogée par Keycloak via le protocole LDAP (port 389/636) ;
- Un **poste client** accédant aux services via un navigateur web.

1.4. Plan d'adressage et nommage

Élément	Valeur
VM Ubuntu Server	192.168.58.130 (NAT — VMnet8) + interface réseau interne

Élément	Valeur
(Docker)	(VMnet1)
VM Windows Server (SRV-AD)	192.168.10.10 /24 (Host-only — VMnet1)
Domaine Active Directory	sso.local (NetBIOS : SSO)
Unité d'organisation fédérée	OU=SSO_Utilisateurs,DC=sso,DC=local
URL du portail SSO	https://keycloak.isi-sso.local:8443
Realm Keycloak dédié	Bienvenue
Grafana	http://192.168.58.130:3000
WordPress	http://192.168.58.130:8081

Le choix d'une **double interface réseau** sur la VM Ubuntu répond à une logique de sécurité : l'interface NAT (VMnet8) fournit l'accès Internet nécessaire au téléchargement des paquets et images, tandis que l'interface host-only (VMnet1) porte les échanges avec le contrôleur de domaine, qui reste ainsi **totalelement isolé d'Internet**.

1.5. Choix techniques structurants

Choix	Justification
Virtualisation VMware Workstation	Reproduction fidèle d'une infrastructure multiserveur sur une seule machine physique ; isolation des environnements Linux et Windows
Conteneurisation Docker / Docker Compose	Déploiement reproductible, isolation des services, gestion déclarative (IaC), redémarrage automatique
PostgreSQL pour Keycloak	Base recommandée en production, en remplacement de la base embarquée de développement
Fédération LDAP en lecture	Keycloak consulte l'annuaire sans jamais le modifier : l'AD

Choix	Justification
seule (READ_ONLY)	reste l'unique source de vérité
OpenID Connect (flux Authorization Code)	Protocole moderne, adapté aux applications web, jetons JWT signés, plus simple que SAML
TLS avec certificat auto-signé	Chiffrement des échanges dès la maquette ; une autorité de certification interne prendra le relais en production
MFA de type TOTP	Standard RFC 6238, compatible avec les applications gratuites (Google/Microsoft Authenticator, FreeOTP), sans coût d'infrastructure

Conclusion

Ce chapitre a dressé le constat des limites de l'authentification décentralisée, spécifié les besoins fonctionnels et non fonctionnels, puis conçu la solution cible à trois niveaux : une architecture conceptuelle en six couches, une cinématique d'authentification fondée sur le flux Authorization Code d'Open ID Connect, et une architecture technique virtualisée et conteneurisée. Tous les éléments sont désormais réunis pour passer à la réalisation concrète de la solution, objet du chapitre suivant.

CHAPITRE V : MISE EN ŒUVRE PRATIQUE ET CONFIGURATION DU SERVEUR SSO

Introduction

Ce chapitre présente la mise en œuvre concrète de la solution conçue au chapitre précédent. La démarche suit un ordre logique : préparation de l'environnement virtualisé (serveurs Ubuntu et Windows), mise en place du contrôleur de domaine Active Directory, validation de l'interconnexion réseau, installation de la plateforme de conteneurisation Docker, déploiement sécurisé du serveur SSO Keycloak, fédération de l'annuaire, puis intégration des applications clientes et sécurisation par authentification multifacteur. Chaque étape est illustrée par des captures d'écran réalisées sur la maquette.

1. Mise en place de l'environnement virtualisé

1.1. Création de la machine virtuelle Ubuntu Server

La première machine virtuelle, nommée `Ubuntu_Server`, héberge l'écosystème conteneurisé (Keycloak, PostgreSQL, Grafana, WordPress). Elle est dotée de 4 Go de mémoire vive, 2 processeurs virtuels et 40 Go de stockage. Conformément à la conception, elle dispose de **deux cartes réseau** : une interface NAT (VMnet8) pour l'accès Internet et une interface host-only (VMnet1) pour la communication avec le contrôleur de domaine.

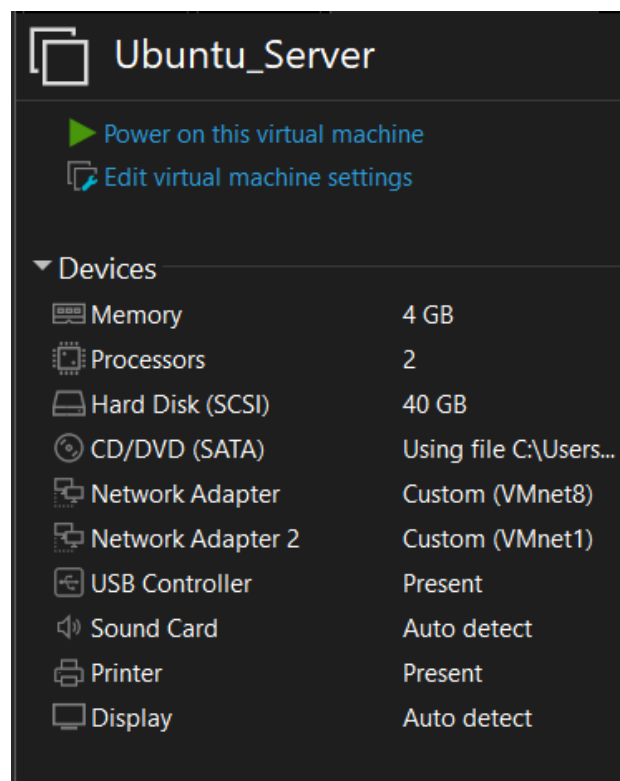


Figure 4.1 : Caractéristiques de la machine virtuelle Ubuntu Server sous VMware

L'installation d'Ubuntu Server s'effectue ensuite via l'assistant officiel. Le profil administrateur du système est configuré comme suit : nom d'hôte `ousmane`, utilisateur `ababacar_ousmane`.

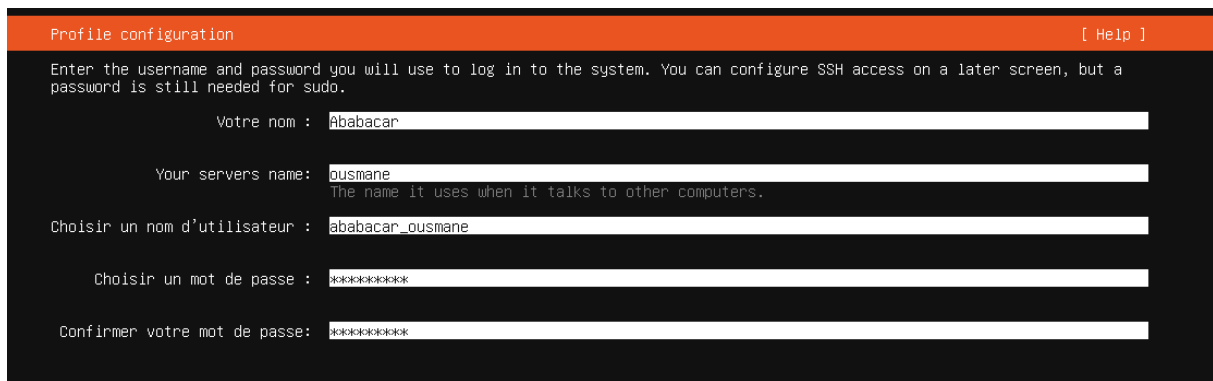


Figure 4.2 : Configuration du profil utilisateur lors de l'installation d'Ubuntu Server

1.2. Création de la machine virtuelle Windows Server

La seconde machine virtuelle, nommée Win_Serv, joue le rôle de contrôleur de domaine. Elle dispose de 4 Go de mémoire vive, 2 processeurs virtuels et 60 Go de stockage. Par choix de sécurité, elle n'est connectée qu'au **réseau interne VMnet1** : le contrôleur de domaine, qui héberge les identités de l'organisation, est ainsi totalement isolé d'Internet.

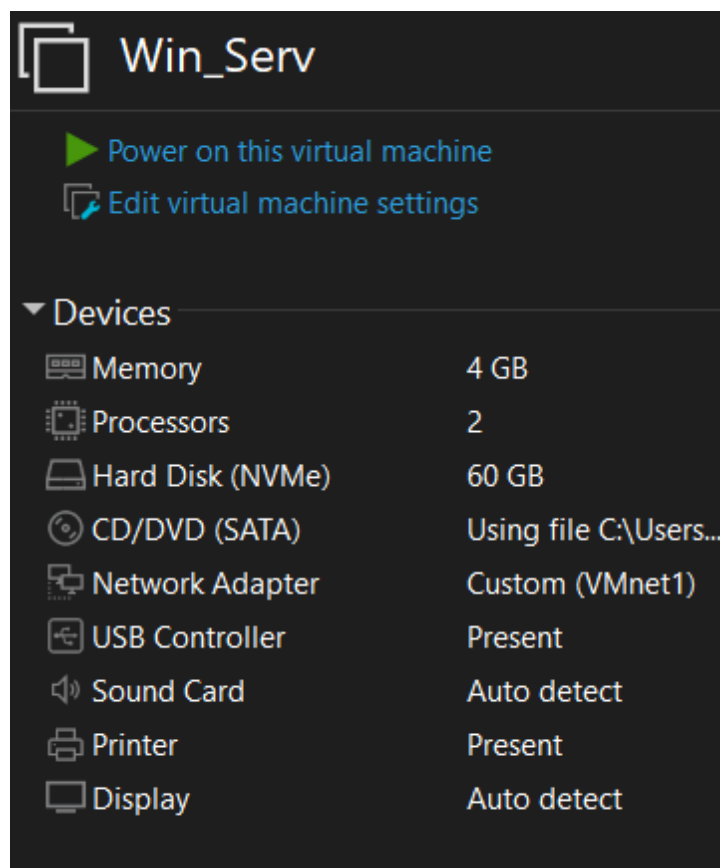


Figure 4.3 : Caractéristiques de la machine virtuelle Windows Server

2. Mise en place du contrôleur de domaine Active Directory

2.1. Configuration réseau statique

Un contrôleur de domaine exige une adresse IP fixe. L'interface du serveur est configurée avec l'adresse statique **192.168.10.10/24**. Le serveur DNS préféré pointe vers **127.0.0.1** : le serveur hébergera en effet son propre service DNS, installé automatiquement avec les services de domaine Active Directory.

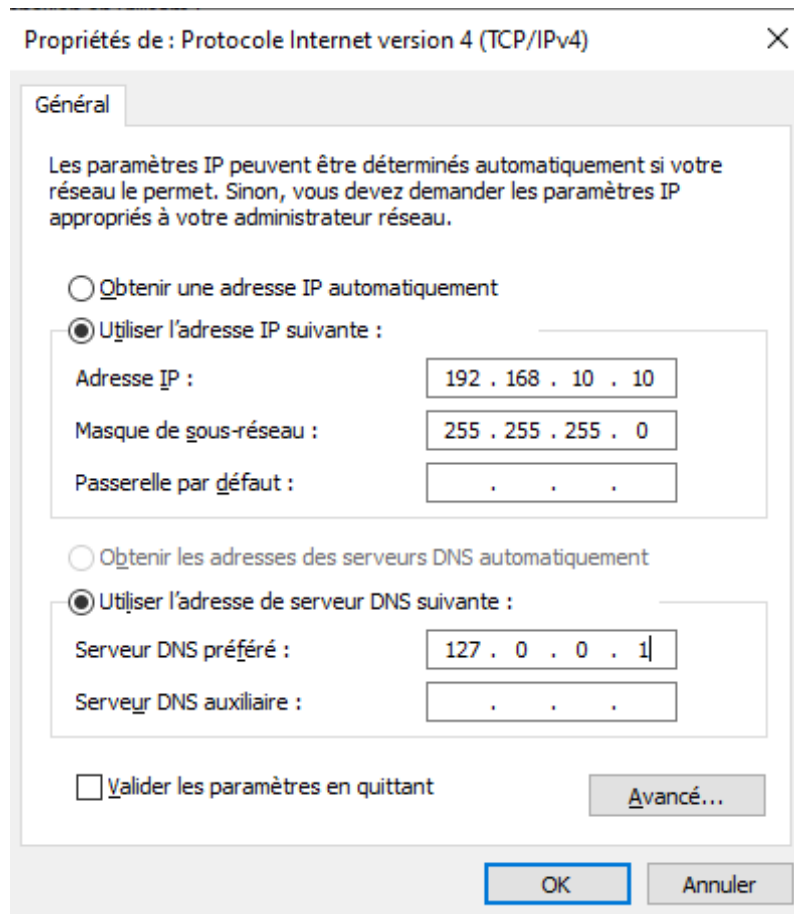


Figure 4.4 : Configuration réseau statique du futur contrôleur de domaine

2.2. Installation du rôle AD DS

Le rôle **Active Directory Domain Services (AD DS)** est installé via l'assistant « Ajout de rôles et de fonctionnalités » du Gestionnaire de serveur, sur le serveur renommé **SRV-AD**. L'installation embarque les outils d'administration associés : Centre d'administration Active Directory, module PowerShell et console de gestion des stratégies de groupe.

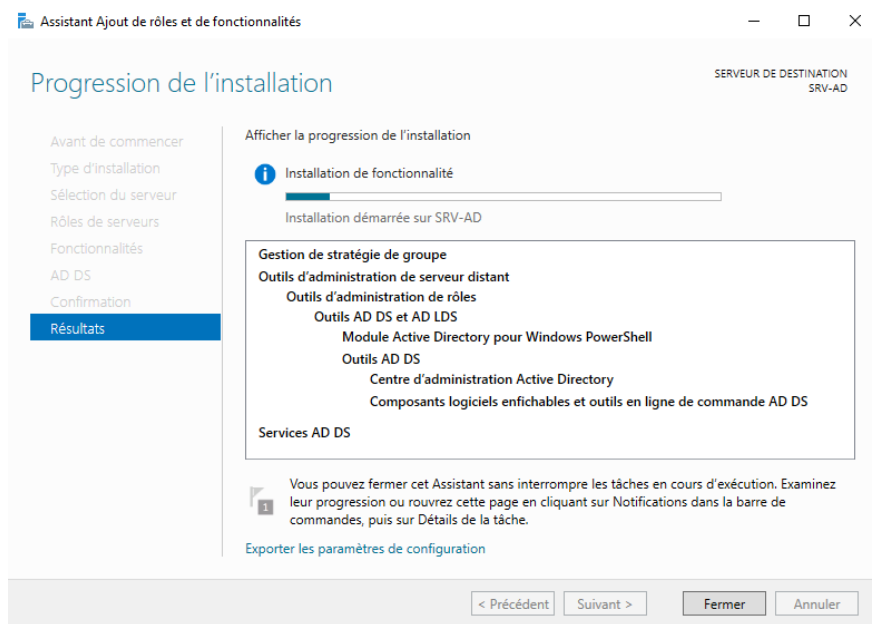


Figure 4.5 : Installation du rôle Active Directory Domain Services sur SRV-AD

À l'issue de l'installation, le serveur est promu contrôleur de domaine d'une nouvelle forêt : le domaine **sso.local** est créé. Après redémarrage, l'ouverture de session s'effectue désormais sur le domaine, comme en atteste la mire de connexion **SSO\Administrateur**.

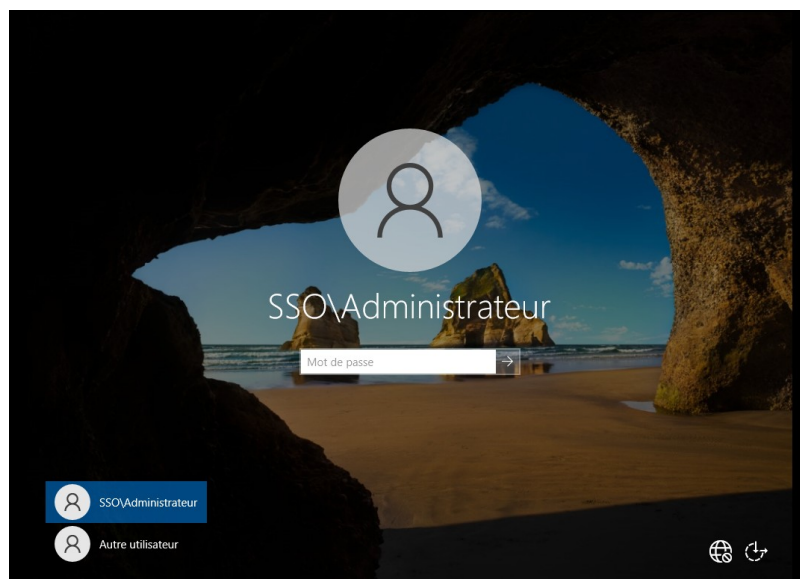


Figure 4.6 : Ouverture de session sur le domaine SSO après promotion du contrôleur

2.3. Création de l'unité d'organisation et des utilisateurs de test

Afin de délimiter précisément le périmètre de la fédération, une unité d'organisation dédiée, **SSO_Utilisateurs**, est créée dans le domaine. Elle accueille les comptes des utilisateurs de test (Bara Ndiaye, Khady Niang, Yacine Niang) qui serviront à valider la chaîne d'authentification de bout en bout.

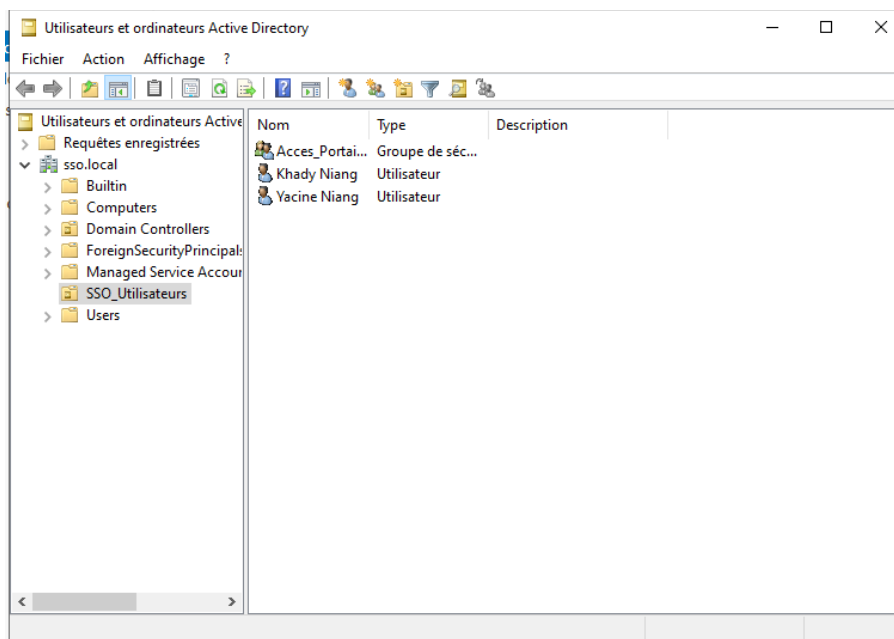


Figure 4.7 : Unité d'organisation SSO_Utilisateurs du domaine sso.local

3. Test de l'interconnexion réseau

Avant tout déploiement applicatif, la connectivité entre les deux serveurs est validée depuis la machine Ubuntu par un test ICMP vers le contrôleur de domaine.

```
ababacar_ousmane@ousmane:~$ ping -c 4 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=128 time=2.02 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=128 time=1.30 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=128 time=1.31 ms
64 bytes from 192.168.10.10: icmp_seq=4 ttl=128 time=0.862 ms

--- 192.168.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 0.862/1.371/2.016/0.413 ms
ababacar_ousmane@ousmane:~$
```

Figure 4.8 : Test de connectivité entre le serveur Ubuntu et le contrôleur de domaine (0 % de perte)

Les quatre paquets émis sont reçus sans perte, avec une latence moyenne inférieure à 2 ms : l'interconnexion des deux réseaux virtuels est opérationnelle. Le serveur Ubuntu pourra donc interroger l'annuaire via LDAP.

4. Expérimentation préliminaire de Keycloak en mode standalone

Une première expérimentation de Keycloak a été menée en mode autonome, directement sur le système hôte. Après vérification de l'environnement d'exécution Java (OpenJDK 21), le serveur Keycloak 24 a été démarré en mode développement.

```
ababacar_ousmane@ousmane:~$ java -version
openjdk version "21.0.11-ea" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11-ea+8-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 21.0.11-ea+8-Ubuntu-1, mixed mode, sharing)
ababacar_ousmane@ousmane:~$
```

Figure 4.9 : Vérification de l'environnement Java (OpenJDK 21)

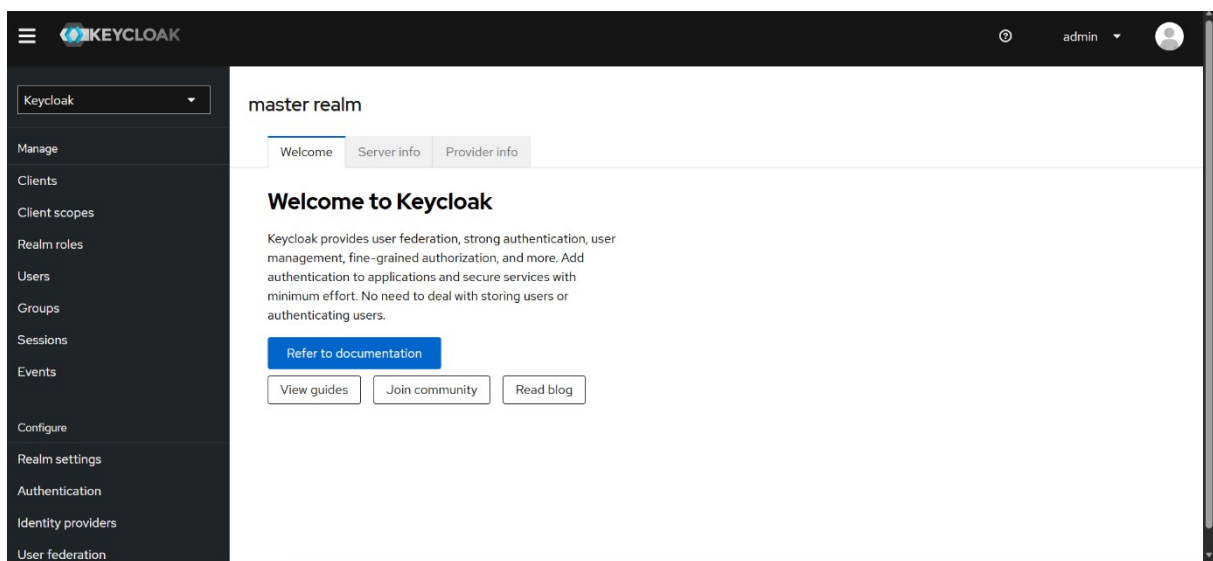


Figure 4.10 : Premier démarrage de Keycloak en mode développement

Cette expérimentation a permis de valider le fonctionnement du serveur et de se familiariser avec sa console. Toutefois, les journaux de démarrage affichent explicitement l'avertissement : « Running the server in development mode. DO NOT use this configuration in production ». Ce mode s'appuie en effet sur une base de données embarquée et n'active pas le chiffrement TLS. La solution a donc été **industrialisée par conteneurisation** : base PostgreSQL dédiée, certificats TLS, redémarrage automatique et isolation réseau — démarche présentée dans les sections suivantes.

5. Installation de la plateforme Docker

5.1. Ajout du dépôt officiel Docker

L'installation suit la procédure officielle pour Ubuntu. La clé GPG du dépôt Docker est d'abord importée afin de garantir l'authenticité des paquets téléchargés.

```

ababacar_ousmane@ousmane:~$ sudo install -m 0755 -d /etc/apt/keyrings
[sudo: authenticate] Password:
ababacar_ousmane@ousmane:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
ababacar_ousmane@ousmane:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg

```

Figure 4.11 : Ajout de la clé GPG officielle de Docker

Le dépôt Docker est ensuite déclaré dans les sources APT du système, puis la liste des paquets est mise à jour : le dépôt `download.docker.com` est bien atteint.

```

ababacar_ousmane@ousmane:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(\
/etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
ababacar_ousmane@ousmane:~$ sudo apt update
Atteint : 1 http://sn.archive.ubuntu.com/ubuntu resolute InRelease
Atteint : 2 http://sn.archive.ubuntu.com/ubuntu resolute-updates InRelease
Atteint : 3 http://sn.archive.ubuntu.com/ubuntu resolute-backports InRelease
Atteint : 4 http://security.ubuntu.com/ubuntu resolute-security InRelease
Atteint : 5 https://download.docker.com/linux/ubuntu resolute InRelease
7 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
ababacar_ousmane@ousmane:~$

```

Figure 4.12 : Déclaration du dépôt Docker et mise à jour des sources APT

5.2. Installation du moteur Docker

Les composants nécessaires sont installés : le moteur (`docker-ce`), le client en ligne de commande (`docker-ce-cli`), l'environnement d'exécution des conteneurs (`containerd.io`) ainsi que les greffons `Buildx` et **Docker Compose**, ce dernier étant indispensable au déploiement déclaratif de la solution.

```

ababacar_ousmane@ousmane:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Installation de :
  containerd.io  docker-buildx-plugin  docker-ce  docker-ce-cli  docker-compose-plugin

Installation de dépendances :
  docker-ce-rootless-extras  pigz

Paquets suggérés :
  cgroupfs-mount | cgroup-lite  docker-model-plugin

Sommaire :
  Mise à niveau de : 0. Installation de : 7, Supprimé : 0. Non mis à jour : 7
  Taille du téléchargement : 99,8 Mo
  Espace nécessaire : 382 Mo / 10,3 Go disponible

Continuer ? [0/n]

```

Figure 4.13 : Installation de Docker Engine et de ses composants

5.3. Vérification du service

Le bon fonctionnement du démon Docker est vérifié : le service est **actif (running)** et activé au démarrage du système.

```

ababacar_ousmane@ousmane:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-09-15 09:34:11 UTC; 22s ago
     Invocation: 85a3fc86b3db44b7a421b6af2fe66fb4
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 79043 (dockerd)
       Tasks: 9
      Memory: 29.5M (peak: 29.7M)
         CPU: 1.580s
      CGroup: /system.slice/docker.service
              └─79043 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

sept. 15 09:34:09 ousmane dockerd[79043]: time="2026-09-15T09:34:09.918360581Z" level=info msg="Restoring containers: start."
sept. 15 09:34:10 ousmane dockerd[79043]: time="2026-09-15T09:34:10.109037700Z" level=info msg="Deleting nftables IPv4 rules" error="running nft: /dev/stdin:
sept. 15 09:34:10 ousmane dockerd[79043]: time="2026-09-15T09:34:10.191350851Z" level=info msg="Deleting nftables IPv6 rules" error="running nft: /dev/stdin:
sept. 15 09:34:11 ousmane dockerd[79043]: time="2026-09-15T09:34:11.485131512Z" level=info msg="Loading containers: done."
sept. 15 09:34:11 ousmane dockerd[79043]: time="2026-09-15T09:34:11.512752983Z" level=info msg="Docker daemon" commit=3ce5872 containerd-snapshotter=true stor
sept. 15 09:34:11 ousmane dockerd[79043]: time="2026-09-15T09:34:11.512231347Z" level=info msg="Initializing buildkit"
sept. 15 09:34:11 ousmane dockerd[79043]: time="2026-09-15T09:34:11.822641097Z" level=info msg="Completed buildkit initialization"
sept. 15 09:34:11 ousmane dockerd[79043]: time="2026-09-15T09:34:11.838422317Z" level=info msg="Daemon has completed initialization"
sept. 15 09:34:11 ousmane dockerd[79043]: time="2026-09-15T09:34:11.838705359Z" level=info msg="API listen on /run/docker.sock"
sept. 15 09:34:11 ousmane systemd[1]: Started docker.service - Docker Application Container Engine.
[Lines 4-23/23 (END)]

```

Figure 4.14 : Vérification du service Docker (état actif)

5.4. Création du réseau dédié à la plateforme SSO

Un réseau Docker nommé **reseau-sso** est créé. Ce pont virtuel isole les flux de la plateforme tout en permettant aux conteneurs (Keycloak, Grafana, WordPress) de communiquer entre eux par résolution de noms interne.

```
ababacar_ousmane@ousmane:~$ sudo docker network create reseau-sso
[sudo: authenticate] Password:
c4f45bebc258c6b55ac9821a75457ab28b18c2abe5b77a55f46cc236670c71a0
ababacar_ousmane@ousmane:~$
```

Figure 4.15 : Création du réseau Docker dédié « reseau-sso »

6. Déploiement du serveur SSO Keycloak

6.1. Arborescence du projet

Le projet est organisé sous `/opt/docker-compose/keycloak`, avec deux sous-répertoires : `certs` (certificats TLS) et `postgresql` (données de la base).

```
ababacar_ousmane@ousmane:~$ sudo mkdir -p /opt/docker-compose/keycloak/certs
ababacar_ousmane@ousmane:~$ sudo mkdir -p /opt/docker-compose/keycloak/postgresql
ababacar_ousmane@ousmane:~$ tree /opt/docker-compose/keycloak
Command 'tree' not found, but can be installed with:
sudo snap install tree # version 2.1.3+pkg-5852, or
sudo apt install tree # version 2.3.1-1
See 'snap info tree' for additional versions.
ababacar_ousmane@ousmane:~$ ls -l /opt/docker-compose/keycloak
total 8
drwxr-xr-x 2 root root 4096 Sep 15 09:56 certs
drwxr-xr-x 2 root root 4096 Sep 15 09:56 postgresql
ababacar_ousmane@ousmane:~$ cd /opt/docker-compose/keycloak
ababacar_ousmane@ousmane:/opt/docker-compose/keycloak$ sudo nano docker-compose.yml
```

Figure 4.16 : Création de l'arborescence du projet Keycloak

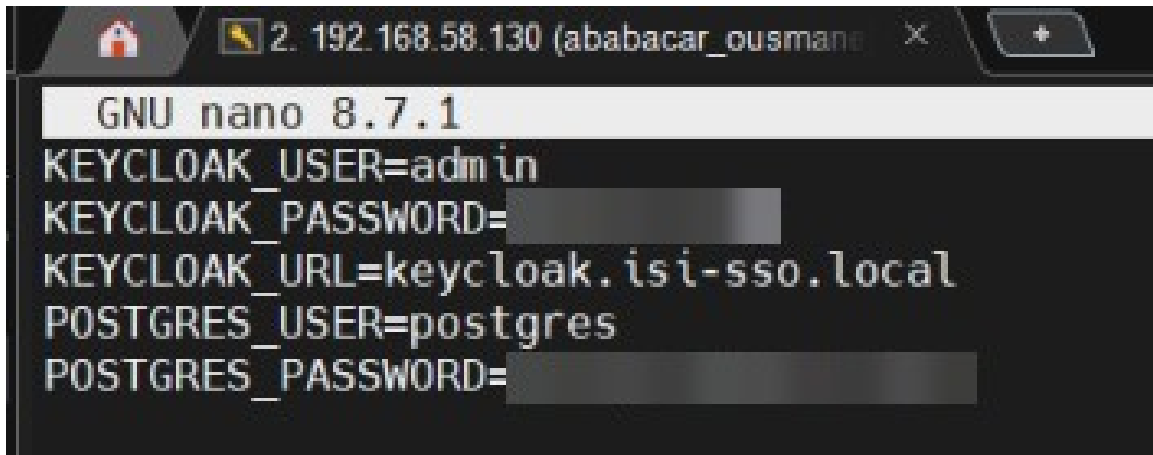
6.2. Variables d'environnement

Les paramètres sensibles (identifiants d'administration, URL du serveur, identifiants de la base de données) sont externalisés dans un fichier `.env`, conformément aux bonnes pratiques : ils ne figurent ainsi jamais en clair dans le fichier de déploiement.

Créez le fichier avec les variables d'environnement à la racine du projet (soit dans le dossier `/opt/docker-compose/keycloak`) :

```
sudo nano .env
```

Figure 4.17 : Création du fichier des variables d'environnement



6.3. Génération du certificat TLS

Afin de chiffrer les échanges dès la maquette, un certificat auto-signé est généré avec OpenSSL : clé RSA de 4 096 bits, validité d'un an, et un nom commun (CN) correspondant au nom de domaine du portail SSO. Les droits de lecture sont ensuite ajustés pour permettre au conteneur d'accéder aux fichiers.

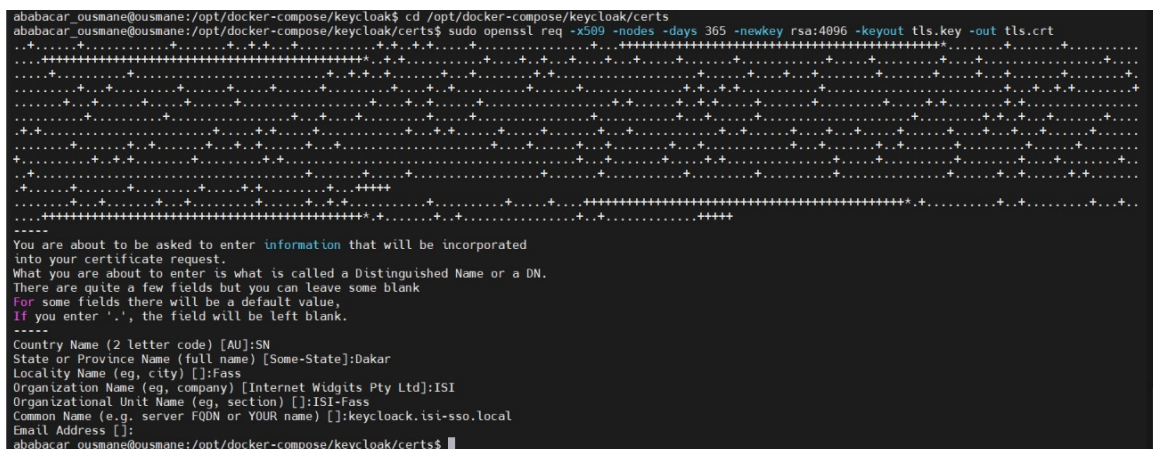


Figure 4.19 : Génération du certificat TLS auto-signé avec OpenSSL (RSA 4096 bits)

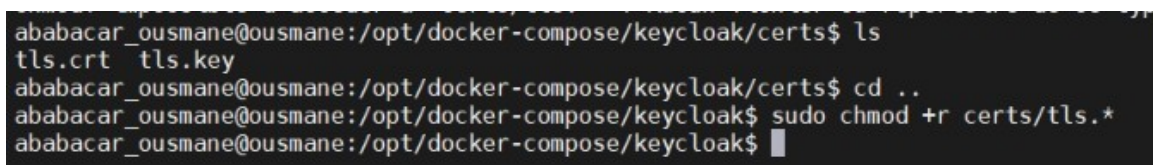


Figure 4.20 : Attribution des droits de lecture sur les certificats TLS

En environnement de production, ce certificat auto-signé serait remplacé par un certificat émis par une autorité de certification interne ou publique.

6.4. Fichier de déploiement Docker Compose

Le fichier docker-compose.yml décrit les deux services : **keycloak_app** (image officielle quay.io, ports 8080/8443, certificats montés en volume) et **keycloak_postgres** (PostgreSQL 17). La directive `depends_on` conditionnée à l'état `healthy` de la base garantit un démarrage ordonné ; la politique `restart: always` assure la reprise automatique des services.

```
GNU nano 8.7.1                                docker-compose.yml *
services:
  keycloak:
    container_name: keycloak_app
    image: quay.io/keycloak/keycloak:latest
    restart: always
    environment:
      KEYCLOAK_ADMIN: ${KEYCLOAK_USER}
      KEYCLOAK_ADMIN_PASSWORD: ${KEYCLOAK_PASSWORD}
      KC_HOSTNAME: ${KEYCLOAK_URL}
      KC_DB: postgres
      KC_DB_USERNAME: ${POSTGRES_USER}
      KC_DB_PASSWORD: ${POSTGRES_PASSWORD}
      KC_DB_URL_HOST: keycloak_postgres
      KC_DB_URL_DATABASE: keycloak
      KC_HTTPS_CERTIFICATE_FILE: /etc/x509/https/tls.crt
      KC_HTTPS_CERTIFICATE_KEY_FILE: /etc/x509/https/tls.key
    volumes:
      - ./certs:/etc/x509/https
    depends_on:
      keycloak_postgres:
        condition: service_healthy
    ports:
      - "8080:8080"
      - "8443:8443"
    networks:
      - keycloak-network
    command:
      - start

  keycloak_postgres:
    container_name: keycloak_postgres
    image: postgres:17
    restart: always
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: ${POSTGRES_USER}
```

Figure 4.21 : Fichier docker-compose.yml de déploiement de Keycloak et PostgreSQL

6.5. Démarrage de la pile

La commande `docker compose up -d` télécharge les images, crée le réseau et démarre les conteneurs : la base PostgreSQL est déclarée **Healthy** et le serveur Keycloak **Started**.

```
ababacar_ousmane@ousmane:/opt/docker-compose/keycloak$ sudo docker compose up -d
[+] up 27/27
✓ Image postgres:17                               Pulled
✓ Image quay.io/keycloak/keycloak:latest           Pulled
✓ Network keycloak-network                         Created
✓ Container keycloak_postgres                      Healthy
✓ Container keycloak_app                          Started
ababacar_ousmane@ousmane:/opt/docker-compose/keycloak$
```

Figure 4.22 : Démarrage réussi des conteneurs Keycloak et PostgreSQL

6.6. Accès à la console et durcissement du compte d'administration

La console d'administration est accessible en HTTPS à l'adresse <https://keycloak.isi-sso.local:8443>.

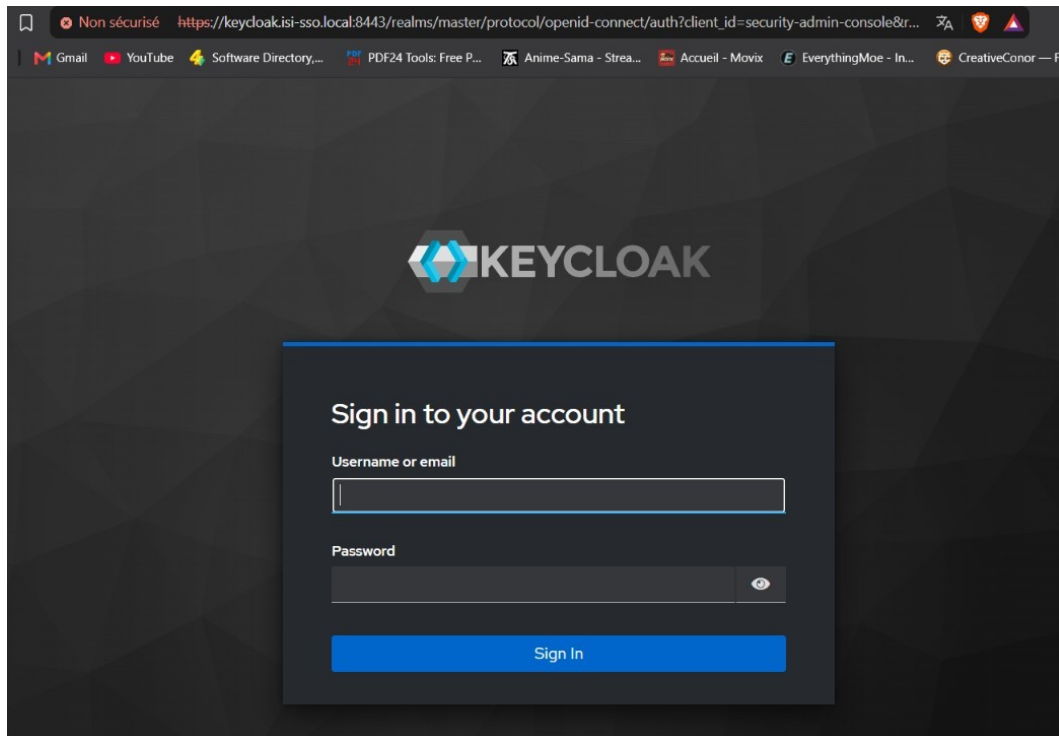


Figure 4.23 : Page d'authentification de la console Keycloak (HTTPS, port 8443)

À la première connexion, Keycloak signale que le compte d'administration initial est temporaire. Conformément à cette recommandation, un compte administrateur permanent **adm_isi** est créé, doté d'un mot de passe non temporaire, puis le rôle **admin** lui est attribué.

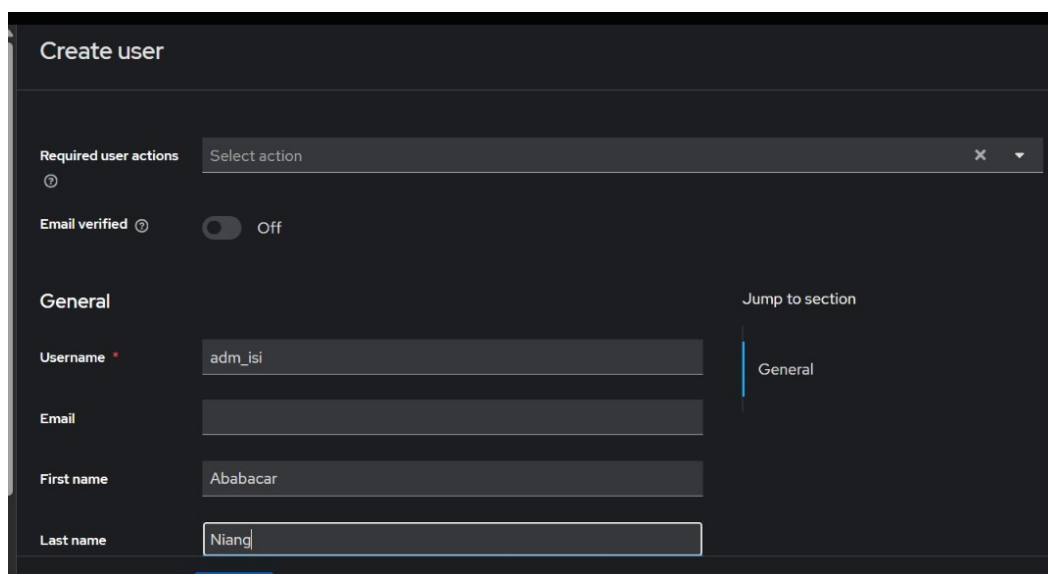


Figure 4.24 : Création du compte administrateur permanent adm_isi

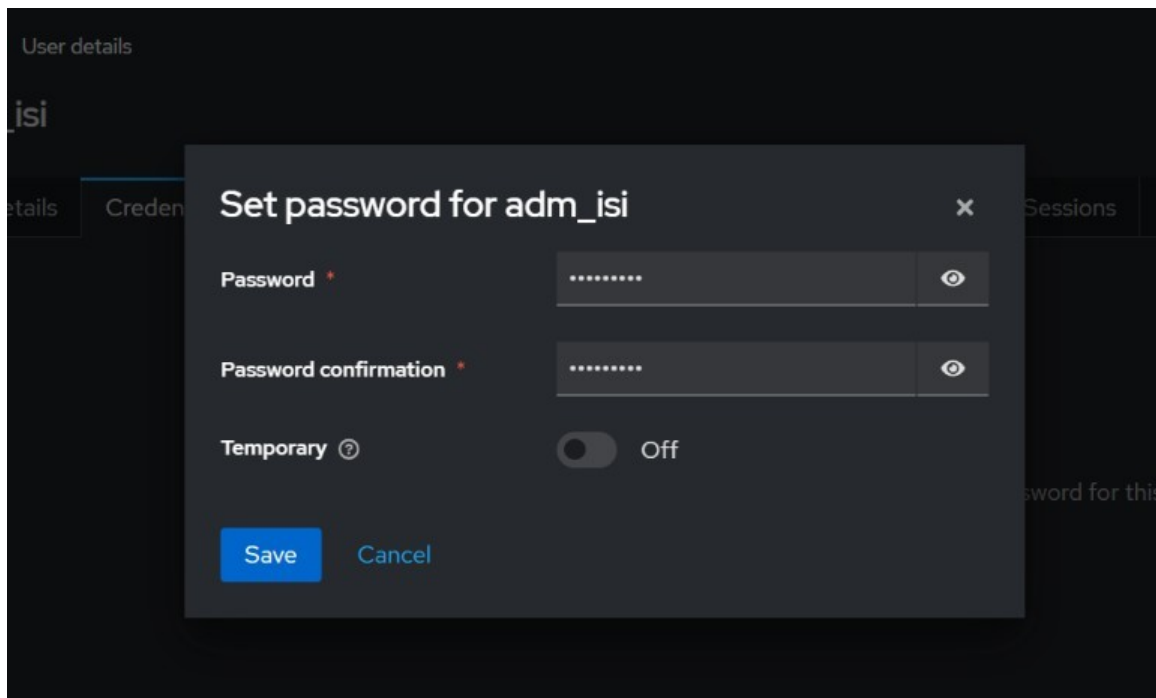


Figure 4.25 : Définition du mot de passe permanent (option « Temporary » désactivée)

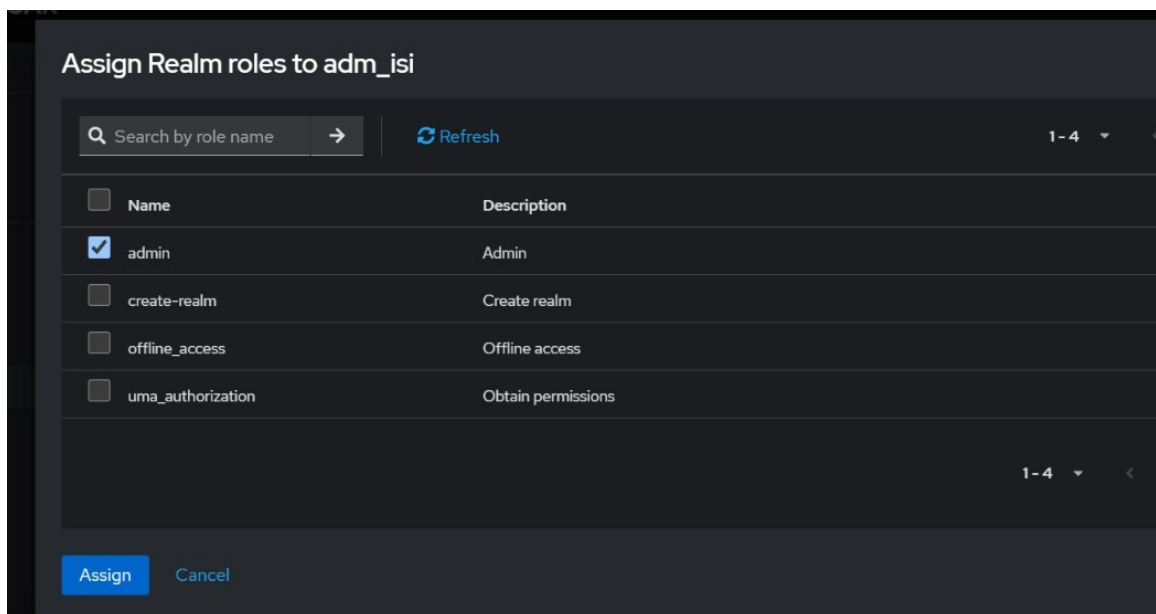


Figure 4.26 : Attribution du rôle admin au compte adm_isi

6.7. Création du realm dédié

Dans Keycloak, un realm est un espace d'administration isolé regroupant utilisateurs, clients et configurations. Le realm **master** étant réservé à l'administration de la plateforme, un realm dédié « **Bienvenue** » est créé pour héberger la fédération et les applications métiers — une séparation conforme aux bonnes pratiques.

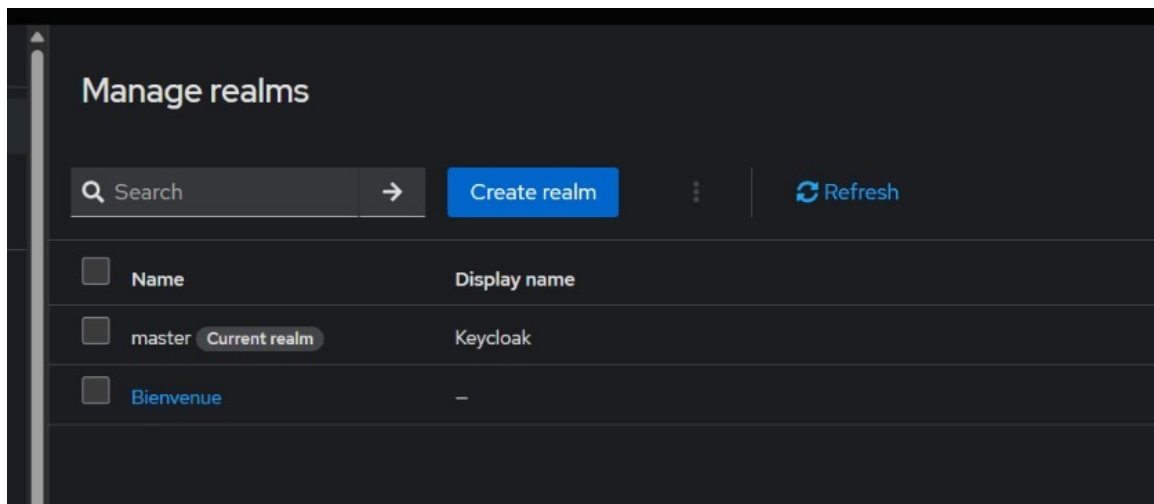


Figure 4.27 : Création du realm dédié « Bienvenue »

7. Fédération des identités avec l'Active Directory

7.1. Configuration de la connexion LDAP

Dans le realm « Bienvenue », un fournisseur de fédération de type LDAP est configuré. La connexion pointe vers le contrôleur de domaine (ldap://192.168.10.10), avec une authentification simple par le compte de service Administrateur@sso.local.

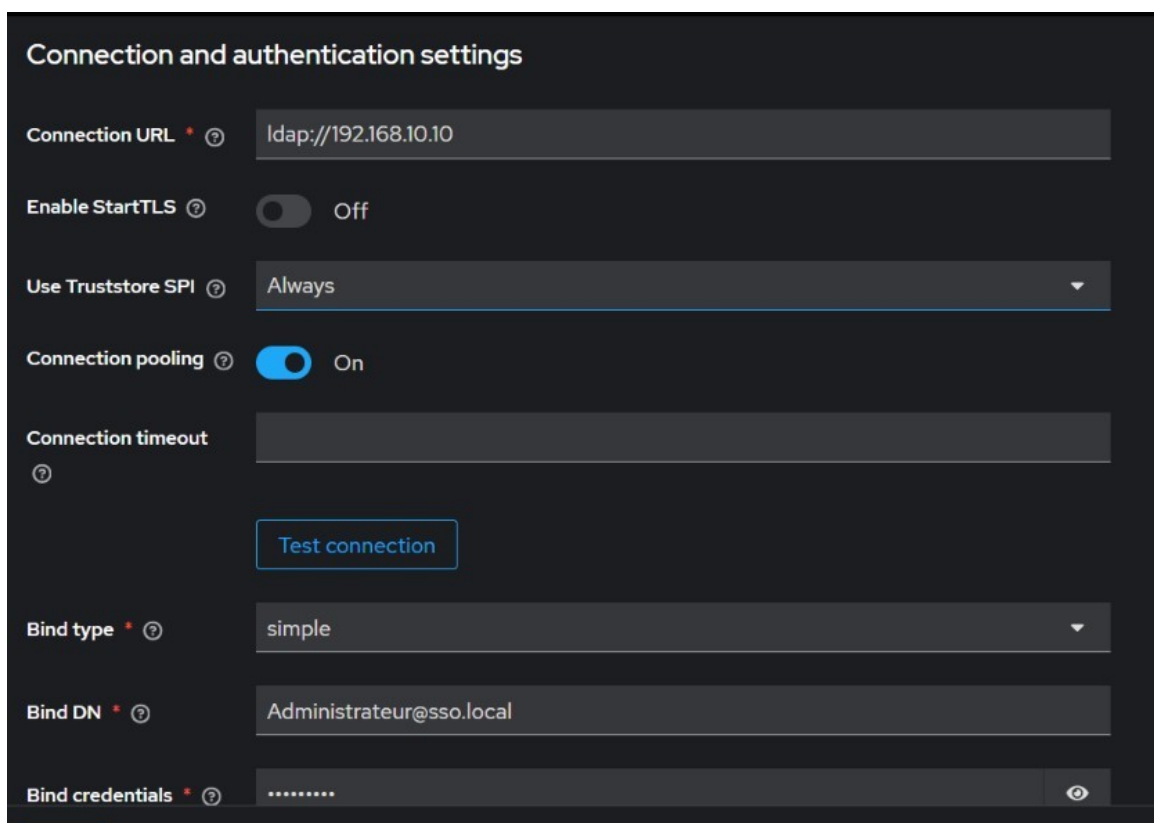
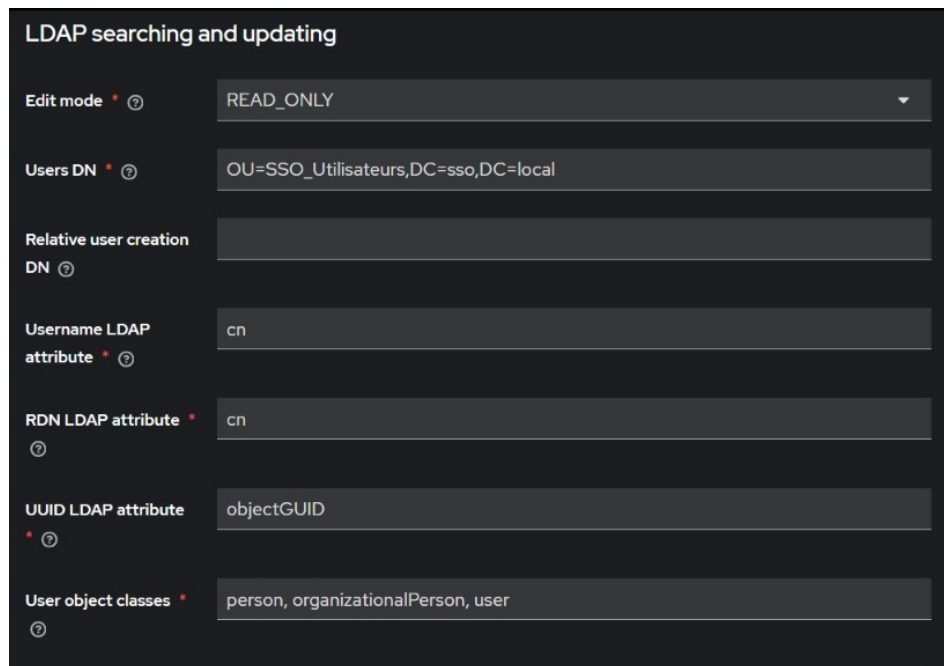


Figure 4.28 : Paramètres de connexion de Keycloak à l'annuaire Active Directory

7.2. Périmètre et attributs de recherche

Le périmètre de recherche est restreint à l'unité d'organisation dédiée : OU=SSO_Utilisateurs,DC=sso,DC=local. Le mode d'édition **READ_ONLY** garantit que Keycloak consulte l'annuaire sans jamais le modifier : l'Active Directory demeure l'unique source de vérité. Les attributs LDAP sont mappés sur le schéma AD (cn, objectGUID, classes person, organizationalPerson, user).



LDAP searching and updating

Edit mode * ⓘ READ_ONLY

Users DN * ⓘ OU=SSO_Utilisateurs,DC=sso,DC=local

Relative user creation DN ⓘ

Username LDAP attribute * ⓘ cn

RDN LDAP attribute * ⓘ cn

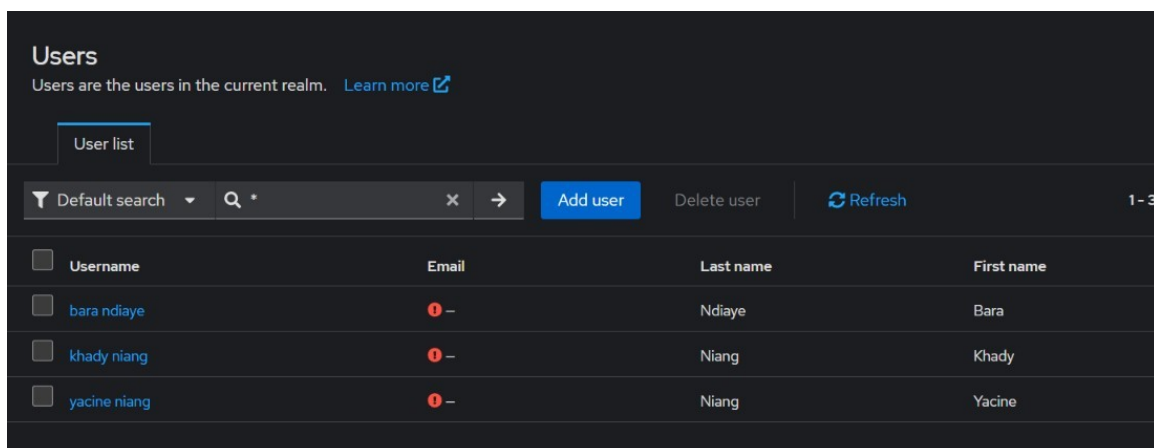
UUID LDAP attribute * ⓘ objectGUID

User object classes * ⓘ person, organizationalPerson, user

Figure 4.29 : Configuration de la recherche des utilisateurs dans l'annuaire (mode lecture seule)

7.3. Validation de la synchronisation

Après synchronisation, les comptes créés dans l'Active Directory apparaissent dans la base d'utilisateurs du realm : **la fédération des identités est opérationnelle**.



Users

Users are the users in the current realm. [Learn more](#)

User list

Default search 🔍 * ✕ → Add user Delete user Refresh 1 - 3

☐ Username	Email	Last name	First name
☐ bara ndiaye	❗ -	Ndiaye	Bara
☐ khady niang	❗ -	Niang	Khady
☐ yacine niang	❗ -	Niang	Yacine

Figure 4.30 : Synchronisation réussie des utilisateurs de l'Active Directory vers Keycloak

Comme l'illustre la figure 4.30, les comptes Bara Ndiaye, Khady Niang et Yacine Niang — créés exclusivement dans l'unité d'organisation SSO_Utilisateurs du domaine sso.local — sont désormais visibles dans Keycloak, confirmant le succès de la fédération.

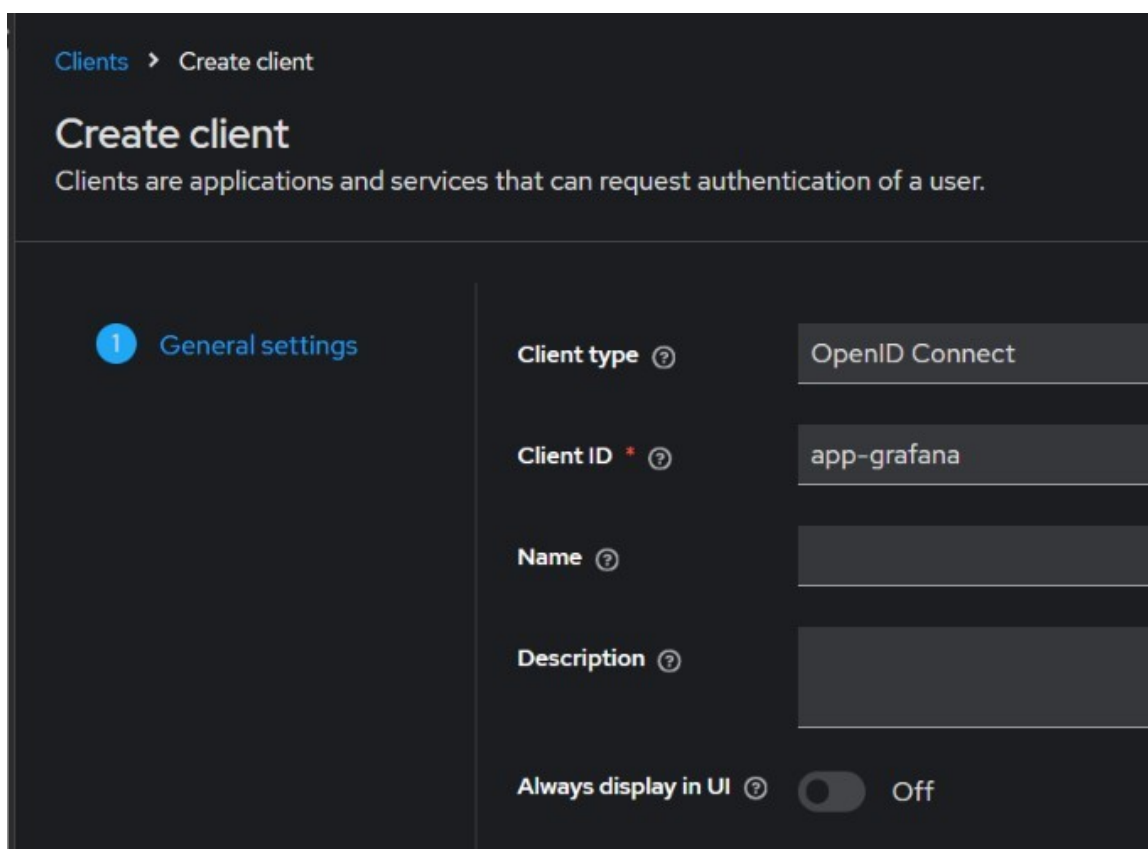
CHAPITRE VI : TESTS DE VALIDATION, SÉCURITÉ (MFA) ET RÉSULTATS

1. Intégration de la première application cliente : Grafana

Grafana, plateforme de visualisation et de supervision largement utilisée en entreprise, constitue la première application intégrée. Elle présente l'avantage de disposer d'un **support natif d'OAuth 2.0/OpenID Connect**, configurable par simples variables d'environnement.

1.1. Déclaration du client dans Keycloak

Toute application souhaitant déléguer son authentification doit préalablement être déclarée en tant que **client** dans le realm. Un client de type **OpenID Connect**, identifié app-grafana, est créé dans le realm « Bienvenue ».



Clients > Create client

Create client

Clients are applications and services that can request authentication of a user.

1 General settings

Client type ⓘ OpenID Connect

Client ID * ⓘ app-grafana

Name ⓘ

Description ⓘ

Always display in UI ⓘ ☐ Off

Figure 4.31 : Création du client app-grafana de type OpenID Connect

L'URI de redirection autorisée est strictement limitée au point d'entrée OAuth de Grafana : `http://192.168.58.130:3000/login/generic_oauth`. Cette restriction empêche tout détournement du code d'autorisation vers une adresse illégitime.

Figure 4.32 : Configuration des URI de redirection du client app-grafana

À l'issue de la création, Keycloak génère un **secret client**, credential partagé qui permettra à Grafana de s'authentifier auprès du serveur SSO lors de l'échange du code d'autorisation contre les jetons.

```
- GF_AUTH_GENERIC_OAUTH_ALLOW_SIGN_UP=true
- GF_AUTH_GENERIC_OAUTH_CLIENT_ID=app-grafana
- GF_AUTH_GENERIC_OAUTH_CLIENT_SECRET=[REDACTED]
- GF_AUTH_GENERIC_OAUTH_SCOPES=openid profile email
- GF_AUTH_GENERIC_OAUTH_AUTH_URL=http://keycloak-isi-cso-local:8443/realms/Bienvenue/protocol/openid-connect/auth
```

Figure 4.33 : Récupération du secret client généré par Keycloak (valeur volontairement masquée)

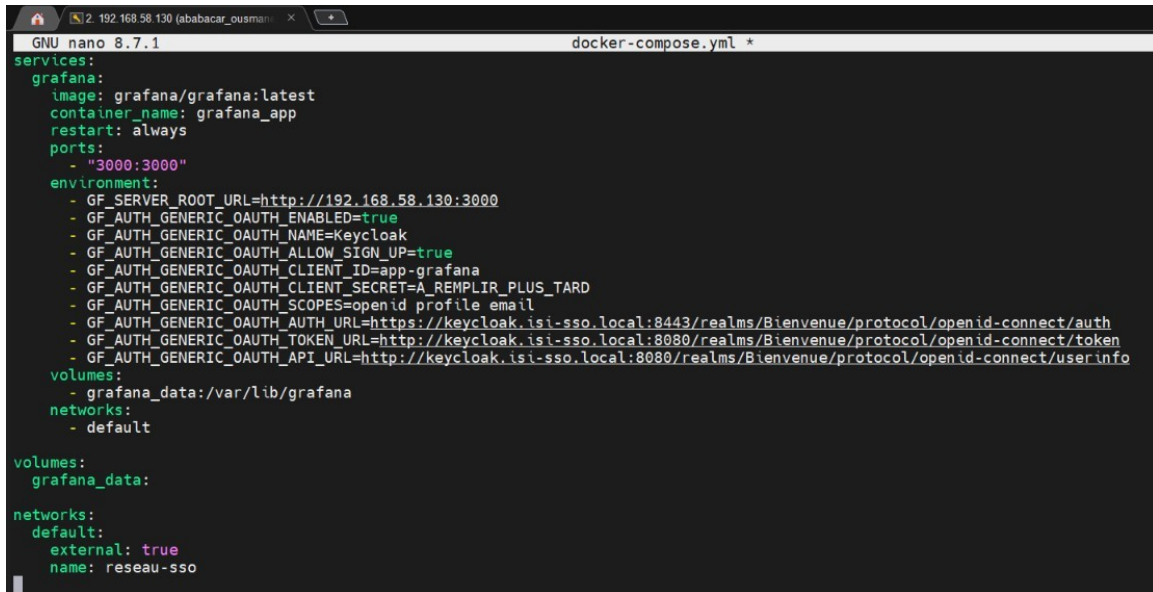
1.2. Déploiement et configuration de Grafana

Le projet est organisé sous `/opt/docker-compose/grafana`.

```
ababacar_ousmane@ousmane:/opt/docker-compose/wordpress$ cd /opt/docker-compose
ababacar_ousmane@ousmane:/opt/docker-compose$ mkdir grafana
mkdir: Permission denied
ababacar_ousmane@ousmane:/opt/docker-compose$ sudo mkdir grafana
ababacar_ousmane@ousmane:/opt/docker-compose$ cd grafana
ababacar_ousmane@ousmane:/opt/docker-compose/grafana$
```

Figure 4.34 : Création de l'arborescence du projet Grafana

Le fichier docker-compose.yml déploie Grafana sur le port 3000 et le rattache au réseau externe `reseau-sso`, lui permettant de joindre Keycloak. L'authentification OAuth générique est activée par les variables `GF_AUTH_GENERIC_OAUTH_*` : identifiant et secret du client, étendues demandées (openid profile email — conformité OpenID Connect), et les trois points de terminaison du realm « Bienvenue » (/auth pour l'autorisation, /token pour l'échange du code, /userinfo pour le profil). L'option `ALLOW_SIGN_UP=true` autorise le **provisioning automatique** : le compte local Grafana est créé à la volée lors de la première connexion fédérée.



```

GNU nano 8.7.1                                docker-compose.yml *
services:
  grafana:
    image: grafana/grafana:latest
    container_name: grafana_app
    restart: always
    ports:
      - "3000:3000"
    environment:
      - GF_SERVER_ROOT_URL=http://192.168.58.130:3000
      - GF_AUTH_GENERIC_OAUTH_ENABLED=true
      - GF_AUTH_GENERIC_OAUTH_NAME=Keycloak
      - GF_AUTH_GENERIC_OAUTH_ALLOW_SIGN_UP=true
      - GF_AUTH_GENERIC_OAUTH_CLIENT_ID=app-grafana
      - GF_AUTH_GENERIC_OAUTH_CLIENT_SECRET=A_REEMPLIR_PLUS_TARD
      - GF_AUTH_GENERIC_OAUTH_SCOPES=openid profile email
      - GF_AUTH_GENERIC_OAUTH_AUTH_URL=https://keycloak.isi-sso.local:8443/realms/Bienvenue/protocol/openid-connect/auth
      - GF_AUTH_GENERIC_OAUTH_TOKEN_URL=http://keycloak.isi-sso.local:8080/realms/Bienvenue/protocol/openid-connect/token
      - GF_AUTH_GENERIC_OAUTH_API_URL=http://keycloak.isi-sso.local:8080/realms/Bienvenue/protocol/openid-connect/userinfo
    volumes:
      - grafana_data:/var/lib/grafana
    networks:
      - default

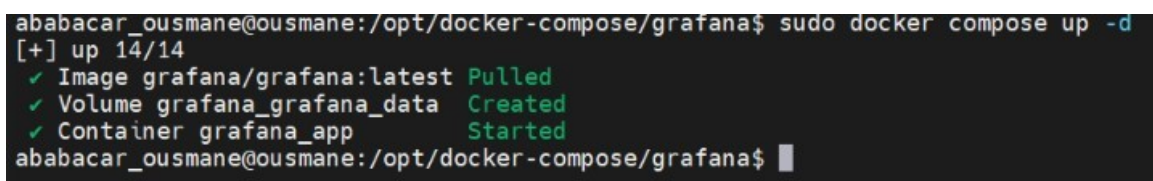
volumes:
  grafana_data:

networks:
  default:
    external: true
    name: reseau-sso

```

Figure 4.35 : Fichier docker-compose.yml de Grafana avec configuration OAuth2/OIDC vers Keycloak

Le conteneur est ensuite démarré.



```

ababacar_ousmane@ousmane:/opt/docker-compose/grafana$ sudo docker compose up -d
[+] up 14/14
✓ Image grafana/grafana:latest Pulled
✓ Volume grafana_grafana_data Created
✓ Container grafana_app Started
ababacar_ousmane@ousmane:/opt/docker-compose/grafana$

```

Figure 4.36 : Démarrage réussi du conteneur Grafana

1.3. Validation de l'authentification SSO sur Grafana

La page de connexion de Grafana propose désormais, en complément de l'authentification locale, le bouton « **Sign in with Keycloak** ».

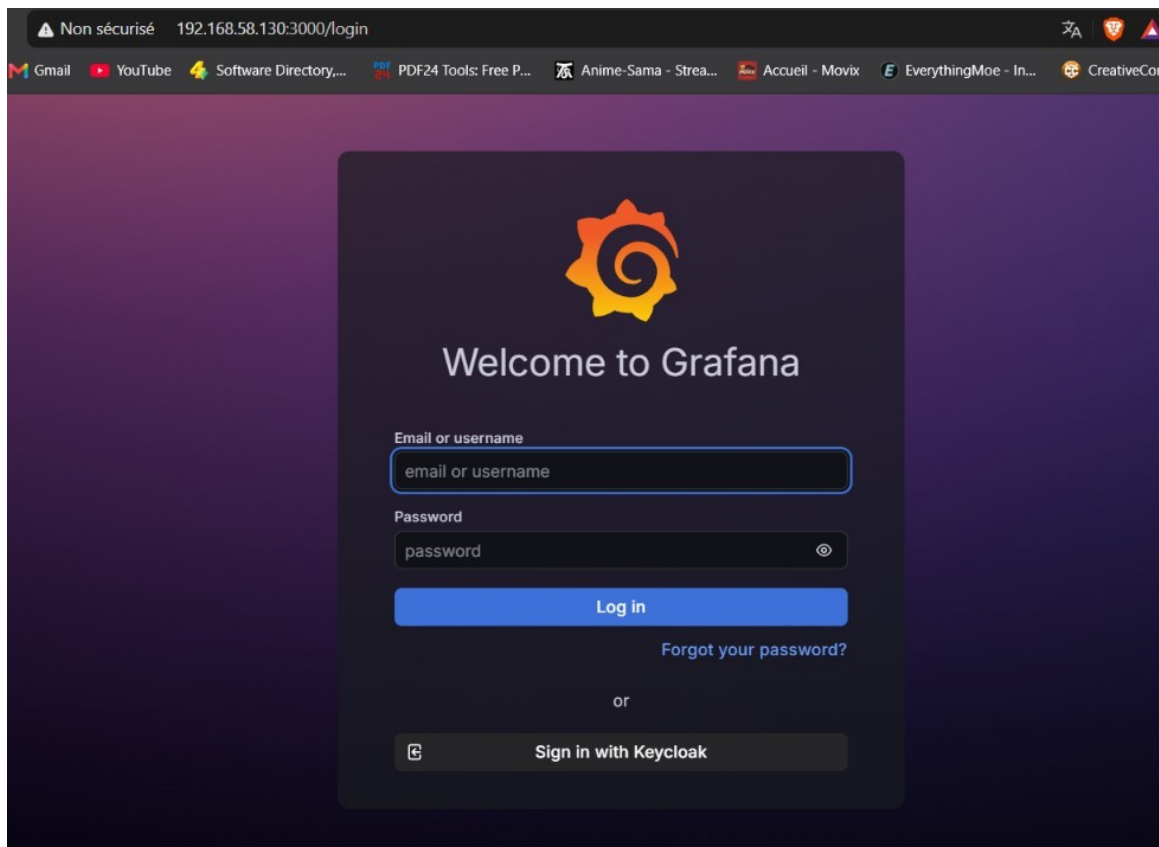


Figure 4.37 : Page d'authentification de Grafana avec l'option de connexion SSO via Keycloak

Après un clic sur ce bouton, l'utilisateur est redirigé vers la mire Keycloak, s'authentifie avec ses identifiants **de domaine**, puis est redirigé vers Grafana où sa session s'ouvre automatiquement.

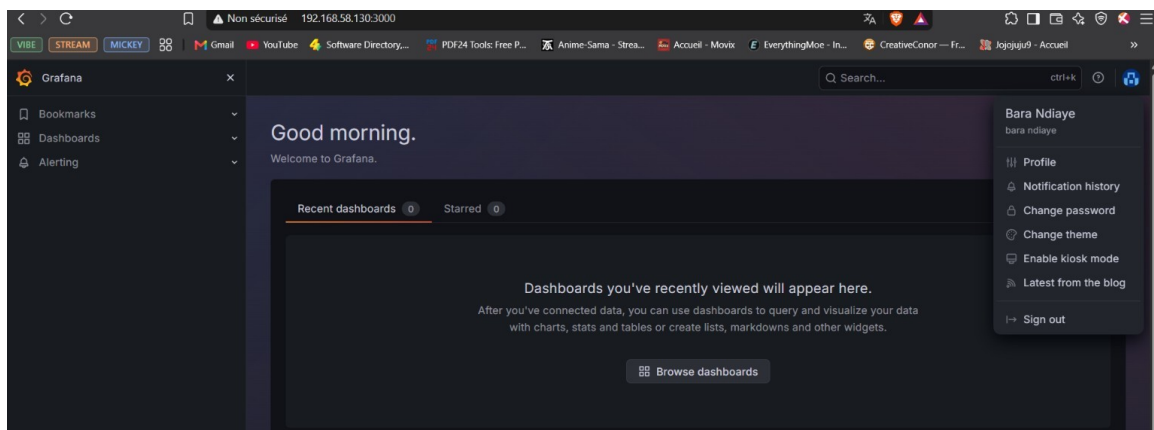


Figure 4.38 : Connexion SSO réussie — session Grafana de l'utilisateur Bara Ndiaye (compte fédéré depuis l'Active Directory)

La figure 4.38 constitue une validation majeure : le compte **Bara Ndiaye** n'a été créé ni dans Grafana ni dans Keycloak — il provient exclusivement de l'annuaire Active Directory.

L'ensemble de la chaîne (fédération LDAP → délégation OIDC → provisioning automatique) fonctionne de bout en bout.

2. Intégration de la seconde application cliente : WordPress

WordPress, système de gestion de contenu le plus répandu au monde, représente un cas d'intégration différent : il **ne dispose pas de support OIDC natif**. Son intégration démontre la capacité de la solution à couvrir également les applications historiques, moyennant un connecteur.

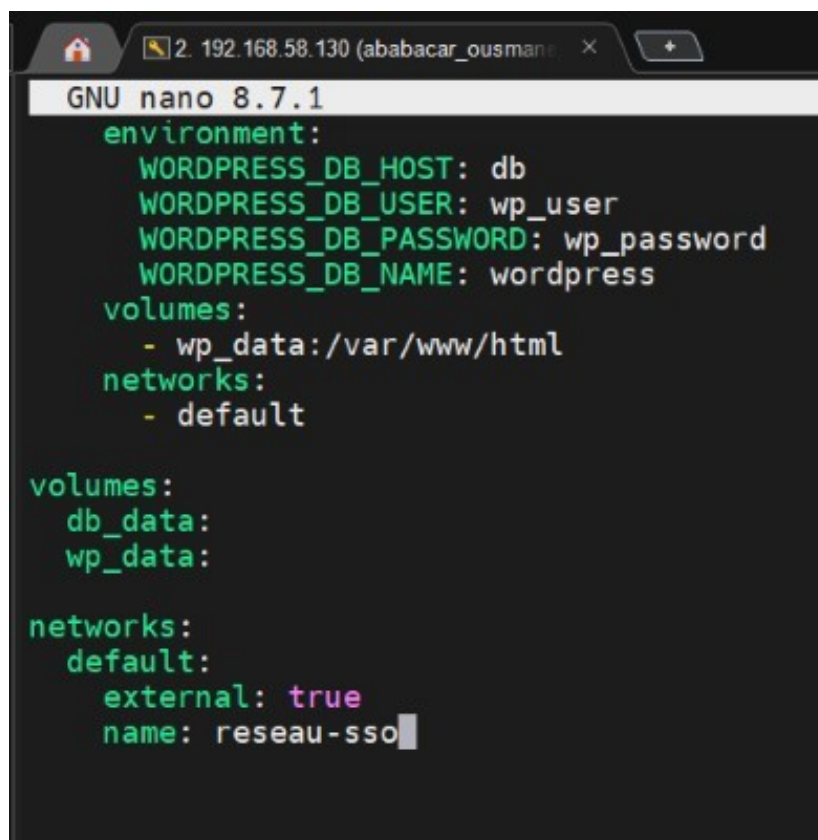
2.1. Déploiement de WordPress

Le projet est organisé sous /opt/docker-compose/wordpress.

```
ababacar_ousmane@ousmane:~$ sudo mkdir -p /opt/docker-compose/wordpress
ababacar_ousmane@ousmane:~$ cd /opt/docker-compose/wordpress
ababacar_ousmane@ousmane:/opt/docker-compose/wordpress$
```

Figure 4.39 : Création de l'arborescence du projet WordPress

Le fichier docker-compose.yml déploie deux services : WordPress (port 8081) et sa base de données MariaDB 10.6, tous deux rattachés au réseau reseau-sso.



```
GNU nano 8.7.1
environment:
  WORDPRESS_DB_HOST: db
  WORDPRESS_DB_USER: wp_user
  WORDPRESS_DB_PASSWORD: wp_password
  WORDPRESS_DB_NAME: wordpress
volumes:
  - wp_data:/var/www/html
networks:
  - default

volumes:
  db_data:
  wp_data:

networks:
  default:
    external: true
    name: reseau-sso
```

Figure 4.40 : Fichier docker-compose.yml de WordPress et de sa base de données MariaDB

```

ababacar_ousmane@ousmane:/opt/docker-compose/wordpress$ sudo docker compose up -d
[+] up 41/41
✓ Image mariadb:10.6      Pulled
✓ Image wordpress:latest  Pulled
✓ Volume wordpress_db_data Created
✓ Volume wordpress_wp_data Created
✓ Container wordpress_db  Started
✓ Container wordpress_app Started
ababacar_ousmane@ousmane:/opt/docker-compose/wordpress$

```

Figure 4.41 : Démarrage réussi des conteneurs WordPress et MariaDB

L'assistant d'installation de WordPress est ensuite exécuté depuis le navigateur : choix de la langue, création du site « ISI-SSO » et du compte administrateur local.

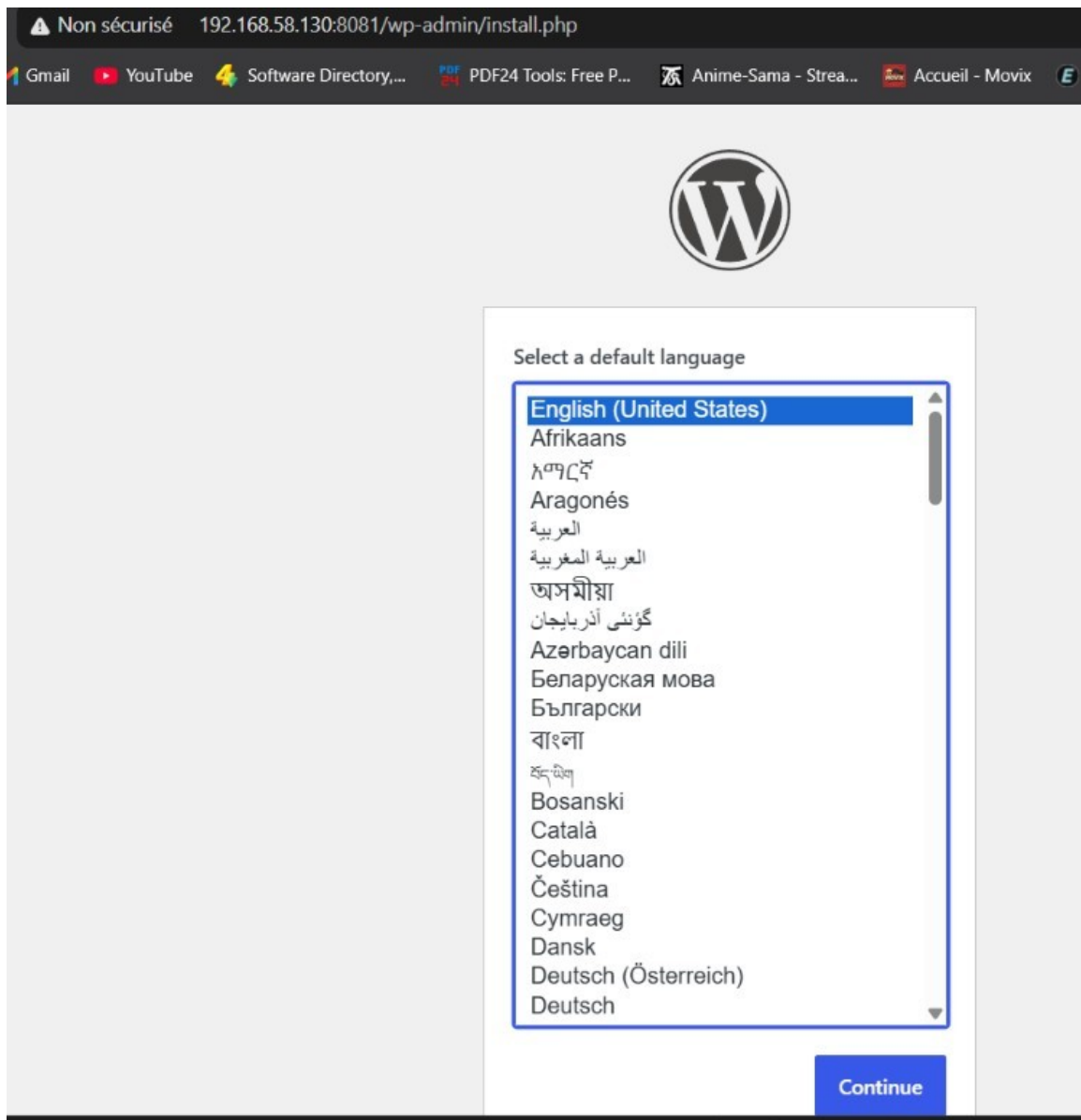


Figure 4.42 : Lancement de l'assistant d'installation de WordPress

Titre du site

Identifiant

Les identifiants ne peuvent utiliser que des caractères alphanumériques, des espaces, des tirets bas ("_"), des traits d'union ("-"), des points et le symbole @.

Mot de passe [Masquer](#)

Mot de passe

Important : Vous aurez besoin de ce mot de passe pour vous connecter. Pensez à le stocker dans un lieu sûr.

Votre e-mail

Vérifiez bien cette adresse e-mail avant de continuer.

Visibilité par les moteurs de recherche ☐ Demander aux moteurs de recherche de ne pas indexer ce site

Certains moteurs de recherche peuvent décider de l'indexer malgré tout.

[Installer WordPress](#)

Figure 4.43 : Configuration du site ISI-SSO et du compte administrateur (valeurs sensibles masquées)

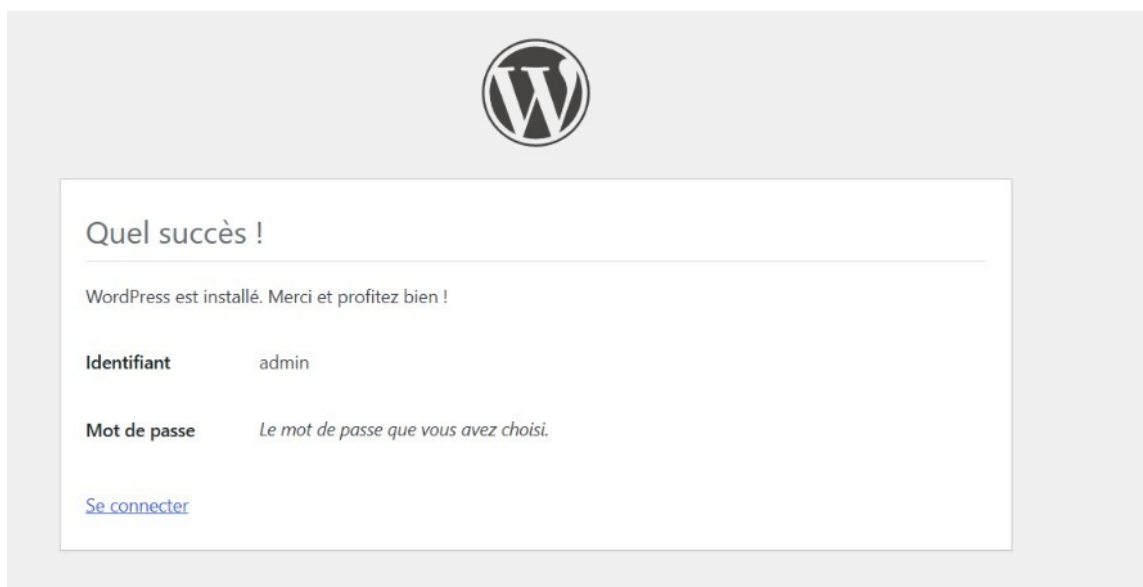


Figure 4.44 : Confirmation de l'installation de WordPress

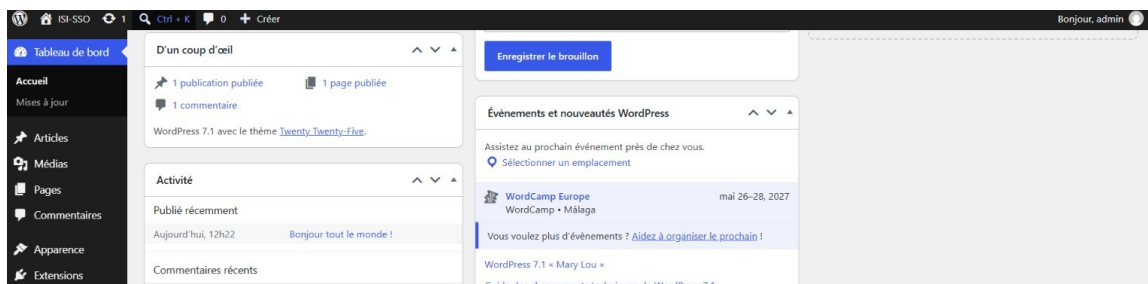


Figure 4.45 : Accès au tableau de bord d'administration de WordPress

2.2. Déclaration du client dans Keycloak

Comme pour Grafana, un client OpenID Connect dédié, app-wordpress, est créé dans le realm « Bienvenue ».

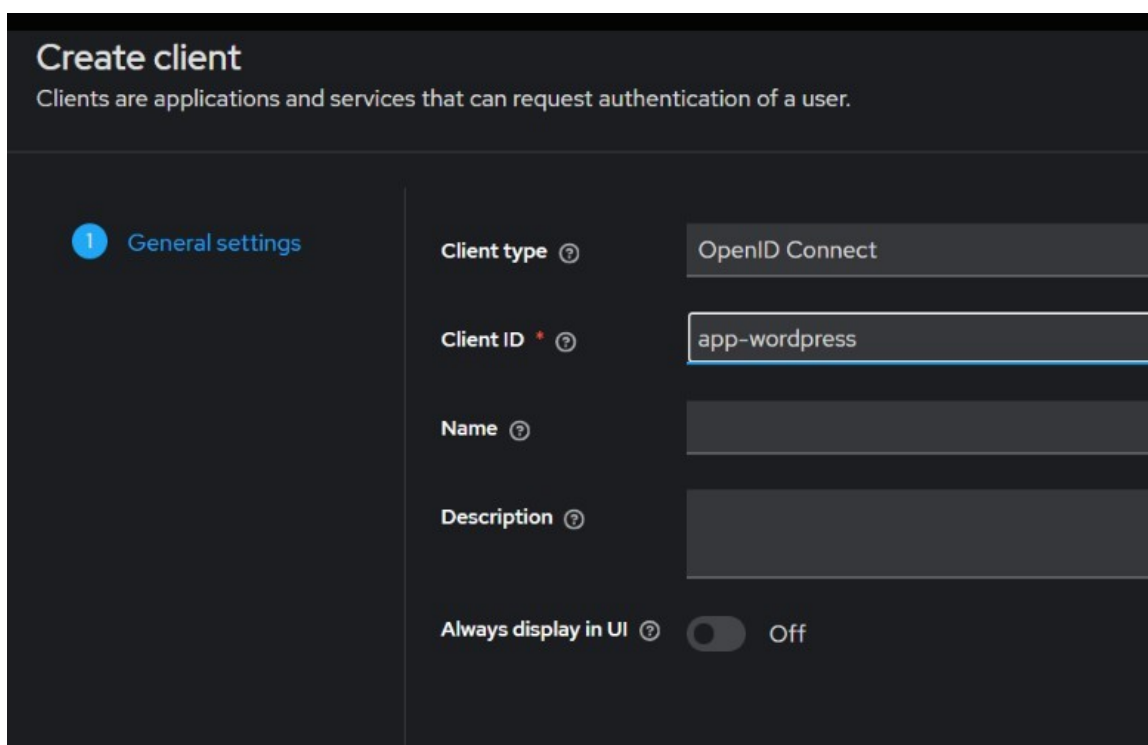


Figure 4.46 : Création du client app-wordpress de type OpenID Connect

2.3. Installation et configuration du connecteur OIDC

L'extension « **OpenID Connect Generic Client** » (plus de 10 000 installations actives) est installée depuis le catalogue officiel d'extensions WordPress.

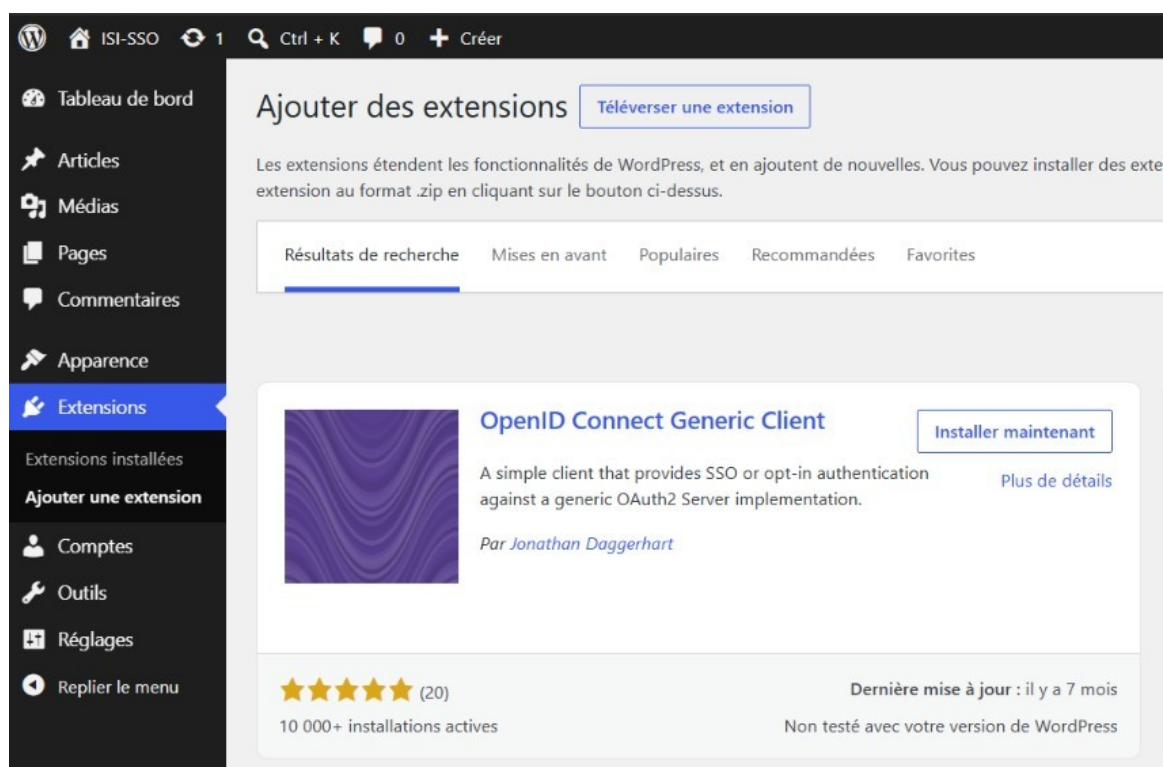


Figure 4.47 : Installation du plugin OpenID Connect Generic Client

Le connecteur est ensuite configuré avec l'identifiant du client, le secret généré par Keycloak, les étendues openid email profile et les points de terminaison du realm (/auth, /userinfo, /token).

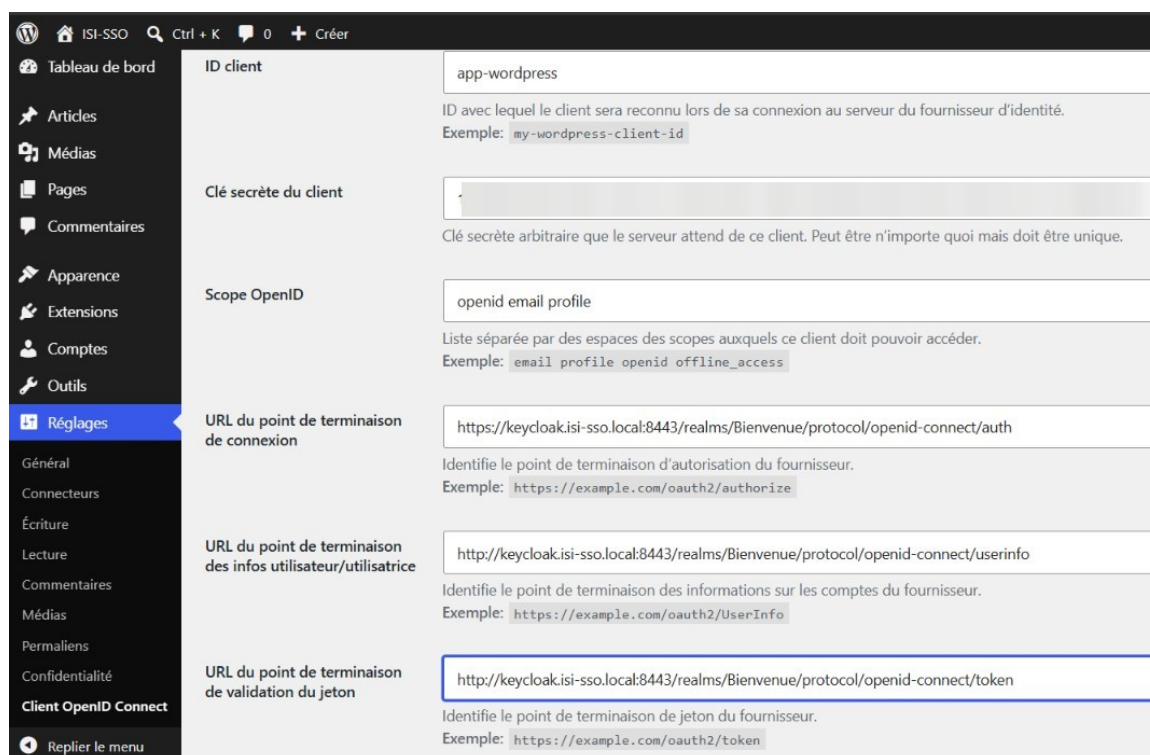


Figure 4.48 : Configuration du plugin OIDC avec les points de terminaison Keycloak (secret masqué)

2.4. Validation

La page de connexion de WordPress propose désormais le bouton « **Connexion avec OpenID Connect** », qui délègue l'authentification à Keycloak.

The image shows the WordPress login page. At the top center is the WordPress logo. Below it is a blue button with the text "Connexion avec OpenID Connect". Underneath the button is a white box with a blue border containing the message "Votre déconnexion a bien été effectuée." Below this is a white login form with a blue border. The form contains two input fields: "Identifiant ou adresse e-mail" and "Mot de passe". The password field has a blue eye icon to its right. Below the password field is a checkbox labeled "Se souvenir de moi" with a question mark icon to its right. A blue button labeled "Se connecter" is positioned to the right of the checkbox. At the bottom of the form is a link "Mot de passe oublié ?". Below the form is a link "← Aller sur ISI-SSO".

WordPress logo

Connexion avec OpenID Connect

Votre déconnexion a bien été effectuée.

Identifiant ou adresse e-mail

Mot de passe

☐ Se souvenir de moi ?

Se connecter

[Mot de passe oublié ?](#)

[← Aller sur ISI-SSO](#)

Figure 4.49 : Page d'authentification WordPress avec l'option de connexion SSO

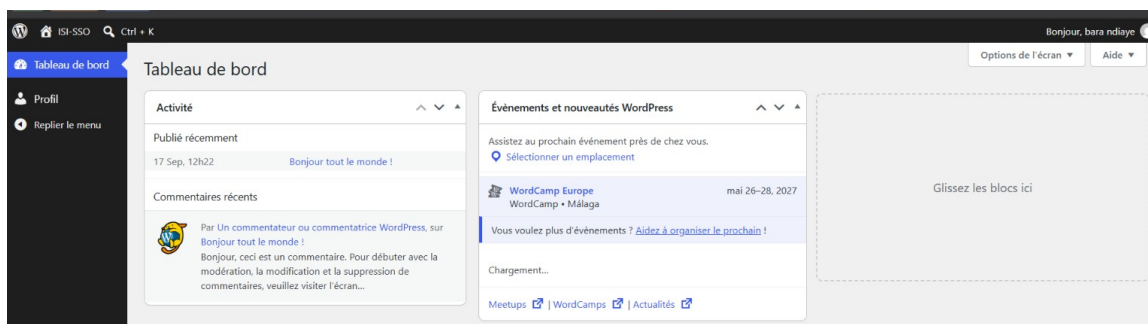


Figure 4.50 : Connexion SSO réussie sur WordPress avec un compte fédéré depuis l'Active Directory

3. Vue d'ensemble de la plateforme déployée

L'état final de la plateforme est résumé par la liste des conteneurs en fonctionnement.

```
ababacar_ousmane@ousmane:/opt/docker-compose/grafana$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
6a681f4b9df8   grafana/grafana:latest              "/run.sh"               2 minutes ago Up 2 minutes   0.0.0.0:3000->3000
/tcp, [::]:3000->3000/tcp
f3a3ab149ae3   wordpress:latest                    "docker-entrypoint.s..." 8 minutes ago Up 8 minutes   0.0.0.0:8081->80/t
cp, [::]:8081->80/tcp
dabc66a15cd4   mariadb:10.6                        "docker-entrypoint.s..." 8 minutes ago Up 8 minutes   3306/tcp
wordpress_db
785fa8537ef    quay.io/keycloak/keycloak:latest    "/opt/keycloak/bin/k..." 17 hours ago  Up 17 hours   0.0.0.0:8080->8080
/tcp, [::]:8080->8080/tcp, 0.0.0.0:8443->8443/tcp, [::]:8443->8443/tcp, 9000/tcp
8801974e80ae    postgres:17                         "docker-entrypoint.s..." 17 hours ago  Up 17 hours (healthy)
keycloak_app
000962758fb0   portainer/portainer-ce:latest       "/portainer"            19 hours ago  Up 17 hours   0.0.0.0:8000->8000
keycloak_postgres
portainer
```

Figure 4.51 : Vue d'ensemble des conteneurs de la plateforme SSO en fonctionnement

Conteneur	Image	Port(s)	Rôle
keycloak_app	quay.io/keycloak/keycloak	8080, 8443	Serveur SSO — fournisseur d'identité
keycloak_postgres	postgres:17	5432	Base de données de Keycloak
grafana_app	grafana/grafana	3000	Application cliente n°1 (OIDC natif)
wordpress_app	wordpress	8081	Application cliente n°2 (connecteur OIDC)
wordpress_db	mariadb:10.6	3306	Base de données de WordPress

Tableau 4.1 : Récapitulatif des services conteneurisés de la plateforme

4. Sécurisation renforcée : authentification multifacteur (MFA)

Conformément aux objectifs du projet, la sécurité des accès est renforcée par une authentification multifacteur de type **TOTP** (Time-based One-Time Password, RFC 6238), activée dans le realm « Bienvenue ».

Le principe : en complément du mot de passe (**facteur de connaissance**), l'utilisateur doit fournir un code à usage unique généré par son téléphone (**facteur de possession**). Ainsi, même en cas de vol du mot de passe, un attaquant ne peut pas accéder au compte.

Lors de sa première connexion après activation de la politique, l'utilisateur est invité à enrôler son appareil mobile : il scanne un QR code avec une application d'authentification (Google Authenticator, Microsoft Authenticator ou FreeOTP), puis valide avec le code à six chiffres généré.

Mobile Authenticator Setup

You need to set up Mobile Authenticator to activate your account.

1. Install one of the following applications on your mobile:

- Google Authenticator
- Microsoft Authenticator
- FreeOTP

2. Open the application and scan the barcode:



[Unable to scan?](#)

3. Enter the one-time code provided by the application and click Submit to finish the setup.

Provide a Device Name to help you manage your OTP devices.

One-time code *

Device Name

☒ Sign out from other devices

Figure 4.52 : Enrôlement de l'authentification forte — configuration de l'application mobile OTP

Une fois l'enrôlement effectué, chaque authentification sur le portail SSO exige la saisie du code temporaire, renouvelé toutes les 30 secondes. Cette mesure protège **simultanément toutes les applications intégrées** — c'est l'un des bénéfices majeurs de la centralisation : le MFA est déployé une seule fois, au niveau du fournisseur d'identité, et non application par application.

5. Synthèse des tests de validation

#	Test	Résultat attendu	Statut
T1	Connectivité Ubuntu ↔ Windows Server	Ping 0 % de perte	Validé (Fig. 4.8)
T2	Démarrage de la pile Keycloak	Conteneurs Healthy/Started	Validé (Fig. 4.22)
T3	Accès console Keycloak en HTTPS	Mire de connexion sur le port 8443	Validé (Fig. 4.23)
T4	Synchronisation LDAP	Utilisateurs AD visibles dans Keycloak	Validé (Fig. 4.30)
T5	Authentification SSO sur Grafana	Session ouverte avec un compte AD	Validé (Fig. 4.38)
T6	Authentification SSO sur WordPress	Session ouverte via OIDC	Validé (Fig. 4.49/4.50)
T7	Authentification multifacteur	Enrôlement OTP et code exigé à la connexion	Validé (Fig. 4.52)

Tableau 4.2 : Synthèse des tests de validation de la solution

Conclusion

Ce chapitre a détaillé la réalisation complète de la solution : préparation de l'environnement virtualisé, promotion du contrôleur de domaine Active Directory, déploiement conteneurisé et sécurisé de Keycloak, fédération de l'annuaire en lecture seule, intégration de deux applications aux profils différents — Grafana (support OIDC natif) et WordPress (connecteur tiers) — et

activation de l'authentification multifacteur. Les tests de validation confirment l'atteinte de l'ensemble des objectifs spécifiques : une authentification unique, adossée aux identités du domaine, ouvre désormais l'accès à l'ensemble des applications de la plateforme.

CONCLUSION GÉNÉRALE

1. Bilan

Ce mémoire avait pour objet l'étude et la mise en place d'une solution d'authentification unique (SSO) fondée sur la fédération des identités, en réponse à une problématique désormais centrale dans les organisations : comment centraliser l'accès à des ressources hétérogènes tout en garantissant un niveau de sécurité élevé et une expérience utilisateur fluide ?

La démarche adoptée a suivi une progression rigoureuse. Le premier chapitre a posé le cadre du travail, réalisé au sein du SIMEN (Système Intégré de Management de l'Éducation Nationale), et mis en évidence les risques induits par la fragmentation des identités numériques : mots de passe affaiblis, surcharge du support technique et surface d'attaque élargie.

Le deuxième chapitre a confronté quatre solutions représentatives du marché — Okta, Authentik, Zitadel et Keycloak — au regard de critères adaptés au contexte d'une administration publique : coût, souveraineté des données, support des protocoles standards et intégration avec un annuaire d'entreprise. Cette étude comparative a abouti au choix motivé de **Keycloak**, solution open source de référence soutenue par Red Hat et la CNCF.

Le troisième chapitre a analysé les limites de l'authentification décentralisée existante, spécifié les besoins fonctionnels et non fonctionnels, puis conçu la solution cible : une architecture en six couches, une cinématique d'authentification fondée sur le flux Authorization Code d'OpenID Connect, et une maquette technique virtualisée associant un serveur Ubuntu conteneurisé et un contrôleur de domaine Windows Server isolé d'Internet.

Le quatrième chapitre a concrétisé cette conception. Le serveur Keycloak a été déployé dans un environnement Docker sécurisé par TLS, adossé à une base PostgreSQL. Les identités de l'annuaire Active Directory (domaine sso.local) ont été fédérées en lecture seule vers Keycloak. Deux applications aux profils volontairement différents ont ensuite été intégrées : Grafana, disposant d'un support OIDC natif, et WordPress, application historique intégrée au moyen d'un connecteur. Enfin, l'authentification multifacteur (TOTP) a été activée au niveau du fournisseur d'identité, protégeant d'un seul geste l'ensemble des applications.

Les tests de validation confirment l'atteinte de tous les objectifs spécifiques fixés : un utilisateur créé uniquement dans l'Active Directory s'authentifie une seule fois auprès du portail SSO et accède à l'ensemble des applications intégrées, avec un second facteur d'authentification. Au-delà de la démonstration technique, ce travail valide la pertinence d'une approche **open source et souveraine** de la gestion des identités : l'organisation conserve le contrôle total de ses données d'identité, sans coût de licence ni dépendance envers un fournisseur cloud étranger.

Sur le plan personnel, ce projet a été l'occasion de mobiliser et d'approfondir un large éventail de compétences : administration système Linux et Windows Server, virtualisation VMware, conteneurisation Docker, protocoles de fédération (OIDC, OAuth 2.0, SAML), sécurité (TLS, MFA) et méthodologie de conduite de projet d'infrastructure.

2. Perspectives

Ce travail ouvre plusieurs pistes d'évolution vers un déploiement de production :

- **Haute disponibilité** : déploiement de Keycloak en grappe (cluster) derrière un répartiteur de charge, avec réplication de la base PostgreSQL, afin d'éliminer tout point de défaillance unique ;
- **Certificats de confiance** : remplacement du certificat auto-signé par des certificats émis par une autorité de certification interne ou publique (Let's Encrypt), et passage au LDAPS (port 636) pour le lien avec l'annuaire ;
- **Extension du périmètre applicatif** : intégration progressive des applications métiers de l'organisation, y compris les services SaaS via SAML 2.0, et des API REST via la validation de jetons JWT ;
- **Gouvernance des accès** : exploitation des groupes Active Directory et des rôles Keycloak pour un contrôle d'accès fin par application (autorisation), complétant l'authentification ;
- **Supervision et audit** : centralisation des journaux d'authentification de Keycloak vers une pile de supervision (par exemple Grafana/Loki ou une solution SIEM) pour la détection d'anomalies ;
- **Authentification sans mot de passe** : à plus long terme, adoption des standards FIDO2/WebAuthn (clés de sécurité, biométrie), nativement supportés par Keycloak ;

- **Orchestration** : migration de Docker Compose vers Kubernetes pour l'élasticité et la résilience à grande échelle.

LISTE DES RÉFÉRENCES *(Aux normes APA 7)*

Docker Inc. (2025). *Install Docker Engine on Ubuntu*. Docker Documentation. <https://docs.docker.com/engine/install/ubuntu/>

Grafana Labs. (2025). *Configure generic OAuth2 authentication*. Grafana Documentation. <https://grafana.com/docs/grafana/latest/setup-grafana/configure-security/configure-authentication/generic-oauth/>

Hardjono, T., & Bradley, J. (2012). *The OAuth 2.0 Authorization Framework* (RFC 6749). Internet Engineering Task Force (IETF). [<https://datatracker.ietf.org/doc/html/rfc6749>] (<https://datatracker.ietf.org/doc/html/rfc6749>)

Keycloak. (2025). *Server Administration Guide*. Documentation officielle Keycloak. <https://www.keycloak.org/documentation>

Microsoft. (2025). *Active Directory Domain Services Overview*. Microsoft Learn. <https://learn.microsoft.com/windows-server>

M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-Based One-Time Password Algorithm* (RFC 6238). Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc6238>

OASIS. (2005). *SAML 2.0 Technical Overview*. OASIS Open. [<https://docs.oasis-open.org>] (<https://docs.oasis-open.org>)

OpenID Foundation. (2025). *OpenID Connect Core 1.0*. OpenID Specifications. [https://openid.net/specs/openid-connect-core-1_0.html](https://openid.net/specs/openid-connect-core-1_0.html)

PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>

WordPress.org. (2025). *OpenID Connect Generic Client*. WordPress Plugin Directory. <https://wordpress.org/plugins/daggerhart-openid-connect-generic/>

ANNEXES (structure proposée)

- **Annexe A** : Fichier docker-compose.yml complet de Keycloak (secrets masqués)
- **Annexe B** : Fichier docker-compose.yml complet de Grafana et WordPress
- **Annexe C** : Fiche de dépôt du sujet de mémoire
- **Annexe D** : Captures complémentaires non insérées dans le corps du texte